L A K B A Y

LAKBAY SALITA - OCTOBER 2026

Step by step: how to create Lakbay Salita

From an empty Unity project to a Windows build:
setup, assets, C# scripts, JSON data, prefabs and scenes

Contents

Before you start 2

What you will build 2

What you need 2

How the guide is organised 2

A note about the editor builder scripts 3

Step 1. Create the Unity project 4

Step 2. Project settings 4

Step 3. Create the folder structure 4

Step 4. Install packages 5

Step 5. Set up Git (optional) 5

Step 6. Fonts 7

Step 7. Sprites 7

Step 8. Audio 9

Step 9. Player data, navigation and streak 10

Step 10. ScreenFit: window limits and 16:9 letterbox 20

Step 11. LevelDataRegistry: where the level files are 24

Step 12. The dictionary files (Library) 27

Step 13. Wordle levels 28

Step 14. Crossword puzzles 33

Step 15. Word Search puzzles 38

Step 16. Phrasebook phrases 45

Step 17. Create the 11 prefabs 50

Step 18. Scene basics (every scene) 51

Step 19. Main menu: `1 Main_menu` 52

Step 20. Language hub: `3 Language_Selection_Menu` 55

Step 21. Wordle: `Wordle - Start` and `Wordle - Play` 55

Step 22. Crossword: `Crossword - Start` and `Crossword - Play` 61

Step 23. Word Search: `WordSearch - Start` and `WordSearch - Play` 62

Step 24. Library: `4 Library_menu` 63

Step 25. Learn cards: `Learn - Cards` 65

Step 26. Phrasebook: Topics, Cards, Builder 67

Step 27. Profile and settings 68

Step 28. Music, button sounds and Tap to hear 70

Step 29. Build Settings and the Windows build 73

Step 30. Test checklist 74

Before you start

This guide shows, in order, how to create **Lakbay Salita** from an empty Unity project to a Windows build: the project settings, folders, sprites, fonts, audio, C# scripts, JSON data, prefabs, scenes and the final build. Every code listing is the **real file** from the finished project (`Assets/Main/Scripts/...`), not a simplified copy.

What you will build

- A 2D word-learning game with three languages (Cebuano, Ilonggo, Tagalog), three puzzle games (Wordle, Crossword, Word Search), Learn cards, a Phrasebook with a Sentence builder, a 63,540-word Library, a Profile, music, sound effects and settings.
- 15 scenes, about 97 scripts, 11 prefabs, JSON data for 126 Wordle levels, 63 crosswords, 63 word searches and 53 phrases.

What you need

Item	Details
Unity Hub and Unity 2021.3.45f2	Install the Windows Build Support module. Other 2021.3 LTS versions should work.
Code editor	Visual Studio Code, Visual Studio or Rider.
Git (optional)	For version history; a .gitignore is given in step 5.
Python 3 (optional)	To generate music, sound effects and JSON data; the scripts are described in steps 8 and 12.
Word data	Dictionary files for the three languages (word, translation, definition). Step 12 shows the format.
Fonts and art	The Roboto font family (free) and background and icon images. Step 7 lists every sprite the game uses.

How the guide is organised

Part	Steps	You end up with
1 Project setup	1 to 5	An empty project with settings, folders, packages and Git
2 Assets	6 to 8	Fonts, sprites and audio imported
3 Core code	9 to 11	Player data, navigation, streak, screen fit, level registry
4 Data	12 to 16	JSON files and C# data classes for every game
5 Prefabs	17	The 11 prefabs
6 Scenes and games	18 to 27	All 15 scenes working
7 Audio code	28	Music, button sounds, Tap to hear
8 Build	29 to 30	Build Settings, Windows build, test checklist

> **Conventions.** Language index everywhere: **0 Cebuano, 1 Ilonggo (Hiligaynon), 2 Tagalog**. Topic (category) index 0 to 6 in the order Animals, Food and drink, Body, Family and people, Home and things, Nature and weather, Numbers. All indexes are 0-based. Scene names are written exactly as in the project, including spaces and hyphens.

A note about the editor builder scripts

Most scenes in this project are **created by editor scripts** (menu Tools > ...) that build real scene objects from code: buttons, cards, text, colours and wiring. They are long (500 to 870 lines each), so they are not printed in this guide; they are in the project at `Assets/Main/Scripts/<module>/Editor/`. In each scene step below you can either run the builder (fast, exact) or build the scene by hand from the hierarchy table shown for it. The runtime scripts, which hold the game logic, **are** printed in full.

Part 1. Project setup

Step 1. Create the Unity project

1. Open Unity Hub > Projects > New project.
2. Choose **Unity 2021.3.45f2** and a **2D (Built-in Render Pipeline)** template. Name it **Lakbay-Salita**.
3. Open the project. In Edit > Project Settings > Player > Other Settings, the colour space of the finished project is **Linear**.
4. Create the folder `Assets/Main` (everything of the game goes inside it).

Step 2. Project settings

Setting (Edit > Project Settings > Player)	Value
Product Name	**Lakbay-Salita** (this becomes the .exe name; leaving the old name gives a wrong file name)
Company Name	DefaultCompany (change as you like)
Version	0.1
Resolution and Presentation > Fullscreen Mode	Windowed
Default Screen Width / Height	1024 x 768
Resizable Window	On
Color Space	Linear

Every Canvas in the game uses these settings: **Render Mode Screen Space - Camera**, **UI Scale Mode Scale With Screen Size**, **Reference Resolution 1920 x 1080**, **Screen Match Mode Expand**. The camera is orthographic with size 5 and a solid dark-blue background. The art is designed on a 16:9 canvas; `ScreenFit` (step 10) deals with other window shapes.

Step 3. Create the folder structure

Create these folders (right-click > Create > Folder). The numbers keep the feature folders in order.

Folder tree

```
Assets/
  Main/
    Data/
      Crossword/ (Puzzles, Settings, Legacy levels)
      Language Dictionary/            cebuano.json, hiligaynon.json, tagalog.json
      Phrasebook/                     phrases_<language>.json
      Word Search/ (Puzzles)
      Wordle/ (Levels, Settings, Legacy words)
    Fonts/ (Roboto)
    Prefabs/ (1 Menu, 2 Wordle, 3 Crossword)
    Resources/
      Audio SFX/                      25 sound effects (.wav)
      Audio Soundtrack/               7 music tracks (.wav)
      Audio Words/ (Cebuano, Ilonggo, Tagalog)   word recordings (optional)
      LevelDataRegistry.asset
    Scenes/
      1 Menu/  2 Wordle/  3 Crossword/  4 Word Search/  6 Learn/  7 Phrasebook/
    Scripts/
      1 Menu/ (Editor)   2 Wordle/ (Editor)   3 Crossword/ (Editor, Gameplay, Level Loader)
      4 Word Search/ (Editor)   5 Library/   6 Learn/ (Editor)   7 Phrasebook/ (Editor)
      Audio/ (Editor)    Core/ (Editor)
    Sprites/ (1 Menu, 1 Menu/Main Menu, 2 Wordle, 3 Crossword, 4 Word Search, 5 Library, 6 Learn,
        Letters)
    TextMesh Pro/                     created by the TMP importer
```

Folders named `Editor` hold scripts that only run in the Unity editor (builders, custom inspectors). Unity excludes them from the build automatically. Folders named `Resources` are loaded by name at runtime with `Resources.Load`.

Step 4. Install packages

1. Window > Package Manager > Unity Registry. Make sure **TextMeshPro 3.0.6** and **Unity UI (ugui) 1.0.0** are installed (they are by default).
2. Import TMP Essentials: Window > TextMeshPro > Import TMP Essential Resources.
3. Add **Newtonsoft Json** (package name `com.unity.nuget.newtonsoft-json`, version 3.2.1). The Crossword puzzle generator scripts use it. In the finished project it arrives through the Unity MCP package, so install it yourself if you do not use that package.
4. Optional: the **Unity MCP** package (`https://github.com/CoplayDev/unity-mcp.git?path=/MCPForUnity#main`) lets an AI assistant drive the editor. The game does not need it.

Step 5. Set up Git (optional)

Create the repository in the project folder and add this `.gitignore` before the first commit:

.gitignore

```
[Ll]ibrary/
[Tt]emp/
[Oo]bj/
[Bb]uild/
[Bb]uilds/
[Ll]ogs/
[Mm]emoryCaptures/

UserSettings/

.vscode/
.idea/

*.csproj
*.unityproj
*.sln
*.user
*.userprefs

.DS_Store

# Test screenshots (large, not part of the game)
/Assets/Screenshots/
/Assets/Screenshots.meta
```

- Create two branches: **main** and **sub**. Work and commit on `sub`, then merge into `main` when a version is good.
- Keep Unity's `.meta` files in Git: they hold each asset's import settings and the script links of scenes.
- Screenshots used for testing are ignored so the repository stays small.

Part 2. Assets

Step 6. Fonts

1. Download the **Roboto** family (Thin, ExtraLight, Light, Regular, Medium, SemiBold, Bold, ExtraBold, Black) and place the `.ttf` files in `Assets/Main/Fonts/Roboto`.
2. For each font: Window > TextMeshPro > Font Asset Creator, source font = the file, Sampling Point Size 90, Padding 9, Atlas 1024 x 1024, Render Mode SDFAA, Atlas Population Mode Dynamic. Click Generate and Save as `Roboto-<Weight> SDF.asset` in `Assets/Main/Fonts`.
3. The TMP default font stays **Liberation Sans SDF** (from the TMP essentials). The builder scripts and scenes assign a Roboto asset to each label, so a label you create by hand should get a Roboto asset in its Font Asset field.

All text in the game is **TextMeshPro - Text (UI)**; the builder scripts pick the Roboto weight for each label (Bold for titles and buttons, Regular for body text).

Step 7. Sprites

Put images in `Assets/Main/Sprites`. For every image select it and set **Texture Type = Sprite (2D and UI)**, **Sprite Mode = Single**, **Pixels Per Unit = 100**, **Generate Mip Maps = off**, **Alpha Is Transparency = on**, max size 2048. These are the settings found in the finished project.

Folder	Files (name and purpose)
1 Menu	back_button, play, library, crossword, word_search, wordle, cebuano, ilonggo, tagalog (card icons and flags), profile_icon, edit_button, delete_button, exit, info Frame, language background, library bamboo background, start_photoshop, two large background images, a checkbox and a teal button image, `red.mat`
1 Menu/Main Menu	main_menu_boat, main_menu_gull, main_menu_map, main_menu_circle_hd, main_menu_gear, main_menu_music_on/off, main_menu_play_triangle, main_menu_ripple, main_menu_sky_gradient, main_menu_wave_crest, main_menu_wave_fill, main_menu_wave_mark
2 Wordle	close-up-puzzle-background (puzzle background), sh_rect_ring-rounded (rounded ring for tiles)
3 Crossword	wooden_jigsaw_background (and a duplicate with a misspelt name)
5 Library	library_circle, library_rounded, library_star
6 Learn	speaker_icon (drawn by the Tap to hear installer if missing)
Letters	Four folders (Highlighted, Normal, Plain, Wrong), each with A.png to Z.png: 104 letter sprites

Procedural sprites. Rounded cards, circles and stars are not image files: `LibraryUiKit` draws them into textures at runtime, and the builders create the gear icon and the speaker icon in code. This keeps the sprite folder small.

Letter sprites and the ImageContainer asset. The letter images are grouped in a ScriptableObject so scripts can ask for a letter's sprite in a given state. Create the script below, then Assets > Create > Data > Image Container and drag the images into the four lists.

Assets/Main/Scripts/Core/ImageContainer.cs

```
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// ImageContainer
/// Reusable ScriptableObject data asset holding letter-to-sprite pairs (e.g. 'A' =>
/// "Alphabet A image.png") for the four common visual states an image/tile can be in:
/// Highlighted, Normal, Plain, and Wrong. Mirrors the same one-script, one-list-per-state
/// pattern as AlphabetData
/// (Assets/Sub/Word_Search/Scripts/ScriptableObjects/AlphabetData.cs), but generic —
/// not tied to the Word Search feature — so it can back any game's letter/image states
/// (Wordle, Crossword, Word Search, etc). Create one via Assets > Create > Data >
/// Image Container, fill in each state's letters/sprites in the Inspector (via the
/// matching ImageContainerDrawer custom editor), then look sprites up at runtime with
/// GetSprite(letter, state).
/// </summary>
[CreateAssetMenu(fileName = "ImageContainer", menuName = "Data/Image Container")]
public class ImageContainer : ScriptableObject
{
    public enum ImageState
    {
        Highlighted,
        Normal,
        Plain,
        Wrong
    }

    [System.Serializable]
    public class ImageEntry
    {
        public char letter;
        public Sprite image;
    }

    public List<ImageEntry> highlighted = new List<ImageEntry>();
    public List<ImageEntry> normal = new List<ImageEntry>();
    public List<ImageEntry> plain = new List<ImageEntry>();
    public List<ImageEntry> wrong = new List<ImageEntry>();

    /// <summary>Looks up the sprite for a given letter in the given state. Returns null if not
        found.</summary>
    public Sprite GetSprite(char letter, ImageState state)
    {
        List<ImageEntry> list = GetList(state);
        ImageEntry entry = list?.Find(e => e.letter == letter);
        return entry != null ? entry.image : null;
    }

    private List<ImageEntry> GetList(ImageState state)
    {
        switch (state)
        {
            case ImageState.Highlighted: return highlighted;
            case ImageState.Normal: return normal;
            case ImageState.Plain: return plain;
            case ImageState.Wrong: return wrong;
            default: return null;
        }
    }
}
```

Step 8. Audio

The project's 7 music tracks and 25 sound effects were **generated by Python scripts** (`BGM Ideas/make_bgm.py`, `SFX Ideas/make_sfx.py`) using bamboo marimba, kulintang gong, bandurria and bell tones, so no recordings or licences are needed. Copy the resulting `.wav` files like this:

Folder	Files	Import settings
Resources/Audio Soundtrack	01_main_menu_lakbay, 02_profile_ako_ni, 03_language_selection_tulay, 04_library_tahimik, 05_wordle_hula, 06_crossword_pahalang_pababa, 07_word_search_pangita	Load Type Streaming, Compression Format Vorbis, Preload Audio Data off
Resources/Audio SFX	ui_click, ui_back, ui_open, ui_close, toggle_on, toggle_off, hint, key_tap_1, key_tap_2, key_tap_3, key_delete, key_enter, word_invalid, tile_flip, tile_correct, tile_present, tile_absent, cw_cell_select, cw_word_complete, ws_select, ws_found, ws_wrong, timer_tick, win_jingle, time_up	Force To Mono, Load Type Decompress On Load, Compression ADPCM, Preload Audio Data on
Resources/Audio Words/<Language>	Optional recordings named after the text in lower case, letters and digits only, spaces as underscores (for example `maayong_buntag.wav`)	Same as sound effects

The clips must be in a `Resources` folder because the scripts load them **by name**, for example `Resources.Load<AudioClip>("Audio SFX/ui_click")`. The scripts that play them are in step 28.

Part 3. Core code

Create each script in `Assets/Main/Scripts/Core` (the path is printed above every listing). The class name must match the file name.

Step 9. Player data, navigation and streak

9.1 PlayerData: the save-file model

A plain serializable class. It becomes `player_data.json` on disk. Language arrays hold one number per language.

Assets/Main/Scripts/Core/PlayerData.cs

```
using System;
using System.Collections.Generic;

/// <summary>
/// PlayerData
/// Plain serializable data model for one player's save data: identity plus per-game
/// level-completion progress. This is the schema persisted to disk by PlayerDatabase.
/// To track a new game's progress later, add one more int field here (following the
/// existing "*Progress" fields) and a matching property in PlayerDatabase.
///
/// The fields below the original three were added for the redesigned Profile screen (avatar, join
    date, solved counts per
/// language, best crossword level, learned words). An older player_data.json without them still loads:
    the missing fields
/// keep their defaults.
/// Language index everywhere: 0 = Cebuano, 1 = Ilonggo / Hiligaynon, 2 = Tagalog.
/// </summary>
[Serializable]
public class PlayerData
{
    public string playerNameOrId = "jeho";

    public int wordleProgress = 0;
    public int crosswordProgress = 0;
    public int wordSearchProgress = 0;

    public int avatarIndex = 0;

    /// <summary>Day the profile was created, as yyyy-MM-dd (filled in on first load).</summary>
    public string joinedDate = string.Empty;

    public int[] wordleByLanguage = new int[3];
    public int[] crosswordByLanguage = new int[3];
    public int[] wordSearchByLanguage = new int[3];

    /// <summary>Highest crossword level (1-10) the player has finished.</summary>
    public int bestCrosswordLevel = 0;

    /// <summary>Every different word the player solved or met in a finished puzzle, saved as
        "languageIndex:WORD".</summary>
    public List<string> learnedWords = new List<string>();
}
```

9.2 PlayerDatabase: loading and saving the profile

A singleton that survives scene loads (`DontDestroyOnLoad`), creates itself on demand, loads the file at startup and saves whenever a value changes. Wordle, Crossword and Word Search call `RecordWordleWin`, `RecordCrosswordWin` and `RecordWordSearchWin`.

Assets/Main/Scripts/Core/PlayerDatabase.cs

```
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using UnityEngine;

/// <summary>
/// PlayerDatabase
/// Singleton persistence manager for the player's save data (PlayerData): loads it from
/// disk on startup, exposes read/write access to identity and each game's progress, and
/// saves back to disk whenever a value changes. Survives scene loads (DontDestroyOnLoad),
/// so any scene (menu, profile screen, or a game itself) can reach it via
/// PlayerDatabase.Instance.
///
/// Instance is created on demand: the Profile scene still contains one, but Wordle and Crossword
/// can record a win even if the player never opened the Profile screen first.
///
/// To add a new game's progress later: add a field to PlayerData, then add a property
/// here following the same pattern as WordleProgress/CrosswordProgress/WordSearchProgress.
///
/// Added for the redesigned Profile screen: avatar, join date, solved counts per language, best
    crossword level and the
/// list of learned words. Wordle and Crossword call RecordWordleWin / RecordCrosswordWin when a puzzle
    is won.
/// ResetAll() starts a fresh profile (Library favorites are not touched). Word Search calls
    RecordWordSearchWin when a puzzle is finished.
/// </summary>
public class PlayerDatabase : MonoBehaviour
{
    private static PlayerDatabase _instance;

    /// <summary>The one PlayerDatabase; created automatically (and kept between scenes) the first time
        it is needed while playing.</summary>
    public static PlayerDatabase Instance
    {
        get
        {
            if (_instance == null && Application.isPlaying)
            {
                var go = new GameObject("PlayerDatabase");
                _instance = go.AddComponent<PlayerDatabase>();
            }
            return _instance;
        }
    }

    [SerializeField]
    private PlayerData data = new PlayerData();

    public PlayerData Data => data;

    private string SavePath => Path.Combine(Application.persistentDataPath, "player_data.json");

    private void Awake()
    {
        if (_instance != null && _instance != this)
        {
            Destroy(gameObject);
            return;
        }

        _instance = this;
        DontDestroyOnLoad(gameObject);
        Load();
    }

    public string PlayerNameOrId
```

```
{
    get => data.playerNameOrId;
    set { data.playerNameOrId = value; Save(); }
}

public int WordleProgress
{
    get => data.wordleProgress;
    set { data.wordleProgress = value; Save(); }
}

public int CrosswordProgress
{
    get => data.crosswordProgress;
    set { data.crosswordProgress = value; Save(); }
}

public int WordSearchProgress
{
    get => data.wordSearchProgress;
    set { data.wordSearchProgress = value; Save(); }
}

public int AvatarIndex
{
    get => data.avatarIndex;
    set { data.avatarIndex = Mathf.Max(0, value); Save(); }
}

/// <summary>Day the profile was created (today if it is not known).</summary>
public DateTime JoinedDate
{
    get
    {
        DateTime parsed;
        if (DateTime.TryParseExact(data.joinedDate, "yyyy-MM-dd", CultureInfo.InvariantCulture,
            DateTimeStyles.None, out parsed))
        {
            return parsed;
        }
        return DateTime.Now.Date;
    }
}

public int BestCrosswordLevel => data.bestCrosswordLevel;

/// <summary>How many different words the player has learned so far.</summary>
public int WordsLearned => data.learnedWords.Count;

/// <summary>Wordle words solved in one language (0 Cebuano, 1 Ilonggo, 2 Tagalog).</summary>
public int WordleSolved(int language)
{
    return language >= 0 && language < data.wordleByLanguage.Length ?
        data.wordleByLanguage[language] : 0;
}

/// <summary>Crossword puzzles finished in one language (0 Cebuano, 1 Ilonggo, 2 Tagalog).</summary>
public int CrosswordSolved(int language)
{
    return language >= 0 && language < data.crosswordByLanguage.Length ?
        data.crosswordByLanguage[language] : 0;
}

/// <summary>Word Search puzzles finished in one language (0 Cebuano, 1 Ilonggo, 2
    Tagalog).</summary>
public int WordSearchSolved(int language)
{
    return language >= 0 && language < data.wordSearchByLanguage.Length ?
        data.wordSearchByLanguage[language] : 0;
```

```
}

/// <summary>Counts one finished Word Search puzzle and remembers every word of it as
    learned.</summary>
public void RecordWordSearchWin(int language, IEnumerable<string> words)
{
    if (language < 0 || language > 2)
    {
        return;
    }
    data.wordSearchProgress++;
    data.wordSearchByLanguage[language]++;
    if (words != null)
    {
        foreach (string word in words)
        {
            AddLearned(language, word);
        }
    }
    Save();
}

/// <summary>Counts one solved Wordle word and remembers the word as learned.</summary>
public void RecordWordleWin(int language, string word)
{
    if (language < 0 || language > 2)
    {
        return;
    }
    data.wordleProgress++;
    data.wordleByLanguage[language]++;
    AddLearned(language, word);
    Save();
}

/// <summary>Counts one finished Crossword puzzle, remembers every answer as learned and updates
    the best level.</summary>
public void RecordCrosswordWin(int language, int level, IEnumerable<string> words)
{
    if (language < 0 || language > 2)
    {
        return;
    }
    data.crosswordProgress++;
    data.crosswordByLanguage[language]++;
    data.bestCrosswordLevel = Mathf.Max(data.bestCrosswordLevel, level);
    if (words != null)
    {
        foreach (string word in words)
        {
            AddLearned(language, word);
        }
    }
    Save();
}

private void AddLearned(int language, string word)
{
    if (string.IsNullOrEmpty(word))
    {
        return;
    }
    string key = language + ":" + word.Trim().ToUpperInvariant();
    if (!data.learnedWords.Contains(key))
    {
        data.learnedWords.Add(key);
    }
}
```

```
    /// <summary>Starts a fresh profile: name, avatar, solved counts, learned words and the day streak
        are cleared. Library favorites stay.</summary>
    public void ResetAll()
    {
        data = new PlayerData();
        data.joinedDate = DateTime.Now.ToString("yyyy-MM-dd", CultureInfo.InvariantCulture);
        StreakTracker.Reset();
        WordSearchProgressStore.Reset();
        WordleProgressStore.Reset();
        CrosswordProgressStore.Reset();
        PhraseProgressStore.Reset();
        Save();
    }

    /// <summary>Loads player_data.json from disk, or starts fresh defaults if none exists
        yet.</summary>
    public void Load()
    {
        if (File.Exists(SavePath))
        {
            string json = File.ReadAllText(SavePath);
            data = JsonUtility.FromJson<PlayerData>(json);
        }
        else
        {
            data = new PlayerData();
        }

        // Older save files (and the first run) miss the newer fields.
        if (data.wordleByLanguage == null || data.wordleByLanguage.Length != 3) data.wordleByLanguage =
            new int[3];
        if (data.crosswordByLanguage == null || data.crosswordByLanguage.Length != 3)
            data.crosswordByLanguage = new int[3];
        if (data.wordSearchByLanguage == null || data.wordSearchByLanguage.Length != 3)
            data.wordSearchByLanguage = new int[3];
        if (data.learnedWords == null) data.learnedWords = new List<string>();
        if (string.IsNullOrEmpty(data.joinedDate))
        {
            data.joinedDate = DateTime.Now.ToString("yyyy-MM-dd", CultureInfo.InvariantCulture);
            Save();
        }
    }

    /// <summary>Writes the current PlayerData to disk as JSON.</summary>
    public void Save()
    {
        string json = JsonUtility.ToJson(data, true);
        File.WriteAllText(SavePath, json);
    }
}
```

9.3 StreakTracker: the day streak

Static helper using PlayerPrefs. `SceneNavigator` calls `RecordPlay()` whenever a game opens.

Assets/Main/Scripts/Core/StreakTracker.cs

```
using System;
using System.Globalization;
using UnityEngine;

/// <summary>
/// StreakTracker
/// Static helper that keeps the player's day-streak in PlayerPrefs (no permissions needed).
/// Call RecordPlay() whenever the player starts a minigame (SceneNavigator does this for Wordle,
/// Crossword and Word Search). A day counts once; playing on consecutive local calendar days grows
/// the streak, and missing a whole day resets it to 1 on the next play.
/// GetStreak() returns the streak to display: it is 0 once more than a day has passed without playing.
/// StreakChip reads it for the Main Menu.
/// </summary>
public static class StreakTracker
{
    private const string CountKey = "streak_count";
    private const string LastPlayKey = "streak_last_play";
    private const string BestKey = "streak_best";
    private const string DateFormat = "yyyy-MM-dd";

    /// <summary>Marks today as played and updates the saved streak.</summary>
    public static void RecordPlay()
    {
        DateTime today = DateTime.Now.Date;
        DateTime last;
        bool hasLast = TryGetLastPlay(out last);

        if (hasLast && last == today)
        {
            return;
        }

        int count = PlayerPrefs.GetInt(CountKey, 0);
        count = hasLast && last == today.AddDays(-1) ? count + 1 : 1;

        PlayerPrefs.SetInt(CountKey, count);
        PlayerPrefs.SetString(LastPlayKey, today.ToString(DateFormat, CultureInfo.InvariantCulture));
        if (count > PlayerPrefs.GetInt(BestKey, 0))
        {
            PlayerPrefs.SetInt(BestKey, count);
        }
        PlayerPrefs.Save();
    }

    /// <summary>The longest streak the player ever reached (never lower than the current
        streak).</summary>
    public static int GetBestStreak()
    {
        return Mathf.Max(PlayerPrefs.GetInt(BestKey, 0), GetStreak());
    }

    /// <summary>Forgets the streak and the best streak (used when the profile is reset).</summary>
    public static void Reset()
    {
        PlayerPrefs.DeleteKey(CountKey);
        PlayerPrefs.DeleteKey(LastPlayKey);
        PlayerPrefs.DeleteKey(BestKey);
        PlayerPrefs.Save();
    }

    /// <summary>The streak to show: 0 if the player has not played today or yesterday.</summary>
    public static int GetStreak()
    {
        DateTime last;
        if (!TryGetLastPlay(out last))
        {
            return 0;
```

```
        }

        DateTime today = DateTime.Now.Date;
        if (last == today || last == today.AddDays(-1))
        {
            return PlayerPrefs.GetInt(CountKey, 0);
        }
        return 0;
    }

    private static bool TryGetLastPlay(out DateTime last)
    {
        return DateTime.TryParseExact(PlayerPrefs.GetString(LastPlayKey, string.Empty), DateFormat,
            CultureInfo.InvariantCulture, DateTimeStyles.None, out last);
    }
}
```

9.4 SceneNavigator and LanguageSelector

`SceneNavigator` is placed on a GameObject named **SceneManager** in every menu scene; buttons call its public methods from their OnClick events. `LanguageSelector` reads the three language Toggles (one ToggleGroup) on the hub. Add `LanguageSelector.cs`, `HubToast.cs` and `SceneNavigator.cs` as printed.

Assets/Main/Scripts/1 Menu/LanguageSelector.cs

```
using UnityEngine;
using UnityEngine.UI;

// Which puzzle-word language is currently selected on the Language Selection scene.
public enum PuzzleLanguage
{
    Cebuano,
    Ilonggo,
    Tagalog
}

// Reads whichever language radio button (Toggle) is currently checked so other
// scripts (e.g. SceneNavigator) know which puzzle set to load. The three toggles
// must share one ToggleGroup (set on the "Language_menu" canvas) so only one of
// them can ever be checked at a time.
public class LanguageSelector : MonoBehaviour
{
    [SerializeField] private Toggle cebuanoToggle;
    [SerializeField] private Toggle ilonggoToggle;
    [SerializeField] private Toggle tagalogToggle;

    public PuzzleLanguage SelectedLanguage
    {
        get
        {
            if (cebuanoToggle != null && cebuanoToggle.isOn) return PuzzleLanguage.Cebuano;
            if (ilonggoToggle != null && ilonggoToggle.isOn) return PuzzleLanguage.Ilonggo;
            return PuzzleLanguage.Tagalog;
        }
    }
}
```

Assets/Main/Scripts/1 Menu/HubToast.cs

```
using TMPro;
using UnityEngine;

/// <summary>
/// HubToast
/// A small pop-up message on the Language Selection hub (for example "Word Search is coming soon").
/// Put it on a GameObject that is inactive by default and assign its text label; calling Show(message)
/// activates it, shows the message, and hides it again after a couple of seconds. Used by
/// SceneNavigator when a minigame has no level scenes configured yet.
/// </summary>
public class HubToast : MonoBehaviour
{
    [SerializeField] private TMP_Text label;
    [SerializeField] private float seconds = 2.2f;

    private float _hideAt;

    public void Show(string message)
    {
        Show(message, seconds);
    }

    /// <summary>Shows the message for a custom number of seconds (for example the Wordle "Get ready"
        lead-in).</summary>
    public void Show(string message, float visibleSeconds)
    {
        if (label != null)
        {
            label.text = message;
        }
        gameObject.SetActive(true);
        _hideAt = Time.unscaledTime + visibleSeconds;
    }

    private void Update()
    {
        if (Time.unscaledTime >= _hideAt)
        {
            gameObject.SetActive(false);
        }
    }
}
```

Assets/Main/Scripts/Core/SceneNavigator.cs

```
using UnityEngine;
using UnityEngine.SceneManagement;

// Centralized scene-navigation helper for the menu flow. Attach one instance to a
// "SceneManager" GameObject in each menu scene and wire UI Button OnClick events to
// its public methods to move between scenes (all scenes must be added to Build Settings).
public class SceneNavigator : MonoBehaviour
{
    [Header("Only needed on the Language Selection scene")]
    [SerializeField] private LanguageSelector languageSelector;
    [SerializeField] private string[] cebuanoWordleScenes;
    [SerializeField] private string[] ilonggoWordleScenes;
    [SerializeField] private string[] tagalogWordleScenes;
    [SerializeField] private string[] cebuanoCrosswordScenes;
    [SerializeField] private string[] ilonggoCrosswordScenes;
    [SerializeField] private string[] tagalogCrosswordScenes;
    [SerializeField] private string[] cebuanoWordSearchScenes;
    [SerializeField] private string[] ilonggoWordSearchScenes;
    [SerializeField] private string[] tagalogWordSearchScenes;
    [Tooltip("Optional pop-up used to say a minigame has no levels yet (see HubToast).")]
    [SerializeField] private HubToast hubToast;

    private const string MainMenuScene = "1 Main_menu";
    private const string ProfileIconScene = "2 Profile Icon";
    private const string LanguageSelectionScene = "3 Language_Selection_Menu";
    private const string LibraryMenuScene = "4 Library_menu";

    public void LoadMainMenu() => SceneManager.LoadScene(MainMenuScene);
    public void LoadProfileIcon() => SceneManager.LoadScene(ProfileIconScene);
    public void LoadLanguageSelectionMenu() => SceneManager.LoadScene(LanguageSelectionScene);
    public void LoadLibraryMenu() => SceneManager.LoadScene(LibraryMenuScene);

    // Wired to the Language Selection scene's wordle_button. Remembers the checked language and opens
        the
    // Wordle start screen (scene "Wordle - Start": pick a category and a level). If that scene is not
        in
    // Build Settings it falls back to loading a random puzzle scene of the checked language.
    public void LoadRandomWordlePuzzle()
    {
        if (languageSelector == null)
        {
            Debug.LogWarning("SceneNavigator: languageSelector is not assigned.");
            return;
        }

        if (Application.CanStreamedLevelBeLoaded(WordleSession.StartScene))
        {
            WordleSession.Language = (int)languageSelector.SelectedLanguage;
            SceneManager.LoadScene(WordleSession.StartScene);
            return;
        }

        string[] pool = languageSelector.SelectedLanguage switch
        {
            PuzzleLanguage.Cebuano => cebuanoWordleScenes,
            PuzzleLanguage.Ilonggo => ilonggoWordleScenes,
            PuzzleLanguage.Tagalog => tagalogWordleScenes,
            _ => null
        };

        if (pool == null || pool.Length == 0)
        {
            Debug.LogWarning("SceneNavigator: no puzzle scenes configured for the selected language.");
            return;
        }

        StreakTracker.RecordPlay();
```

```
        SceneManager.LoadScene(pool[Random.Range(0, pool.Length)]);
    }

    // Wired to the Language Selection scene's crossword_button. Remembers the checked language and
        opens the
    // Crossword start screen (scene "Crossword - Start": pick a category and a size). If that scene is
        not in
    // Build Settings it falls back to loading a random crossword scene (Assets/Main/Scenes/3
        Crossword/<N> -
    // Puzzle - <Language>) of the checked language.
    public void LoadRandomCrosswordPuzzle()
    {
        if (languageSelector == null)
        {
            Debug.LogWarning("SceneNavigator: languageSelector is not assigned.");
            return;
        }

        if (Application.CanStreamedLevelBeLoaded(CrosswordSession.StartScene))
        {
            CrosswordSession.Language = (int)languageSelector.SelectedLanguage;
            SceneManager.LoadScene(CrosswordSession.StartScene);
            return;
        }

        string[] pool = languageSelector.SelectedLanguage switch
        {
            PuzzleLanguage.Cebuano => cebuanoCrosswordScenes,
            PuzzleLanguage.Ilonggo => ilonggoCrosswordScenes,
            PuzzleLanguage.Tagalog => tagalogCrosswordScenes,
            _ => null
        };

        if (pool == null || pool.Length == 0)
        {
            Debug.LogWarning("SceneNavigator: no crossword scenes configured for the selected
                language.");
            return;
        }

        StreakTracker.RecordPlay();
        SceneManager.LoadScene(pool[Random.Range(0, pool.Length)]);
    }

    // Wired to the Language Selection scene's Word Search card. Remembers the checked language and
        opens
    // the Word Search start screen (scene "WordSearch - Start": pick a category, puzzle and mode). If
        that
    // scene is not in Build Settings it falls back to the old per-language scene lists, and finally to
        a
    // "coming soon" message.
    public void LoadRandomWordSearchPuzzle()
    {
        if (languageSelector == null)
        {
            Debug.LogWarning("SceneNavigator: languageSelector is not assigned.");
            return;
        }

        if (Application.CanStreamedLevelBeLoaded(WordSearchSession.StartScene))
        {
            WordSearchSession.Language = (int)languageSelector.SelectedLanguage;
            SceneManager.LoadScene(WordSearchSession.StartScene);
            return;
        }

        string[] pool = languageSelector.SelectedLanguage switch
        {
            PuzzleLanguage.Cebuano => cebuanoWordSearchScenes,
```

```
            PuzzleLanguage.Ilonggo => ilonggoWordSearchScenes,
            PuzzleLanguage.Tagalog => tagalogWordSearchScenes,
            _ => null
        };

        if (pool == null || pool.Length == 0)
        {
            Debug.LogWarning("SceneNavigator: no Word Search scenes configured for the selected
                language.");
            if (hubToast != null)
            {
                hubToast.Show("Word Search is coming soon");
            }
            return;
        }

        StreakTracker.RecordPlay();
        SceneManager.LoadScene(pool[Random.Range(0, pool.Length)]);
    }
}
```

Step 10. ScreenFit: window limits and 16:9 letterbox

`ScreenFit` needs no scene object: `RuntimeInitializeOnLoadMethod` creates it before the first scene. It sets every camera's `rect` to the biggest 16:9 rectangle, draws dark bars with a second camera, limits the window to 800 x 600 to 1024 x 768 and switches to a 1920 x 1080 full screen window when the window is maximized (Windows build only, through the `user32` functions `GetActiveWindow` and `IsZoomed`).

Assets/Main/Scripts/Core/ScreenFit.cs

```
using System;
using System.Runtime.InteropServices;
using UnityEngine;
using UnityEngine.SceneManagement;

/// <summary>
/// ScreenFit
/// Keeps the game the right size on any window or screen. It creates itself before the first scene
    loads (RuntimeInitializeOnLoadMethod) and
/// survives scene loads, so nothing has to be placed in a scene. It does two things:
///  1. Window limits (Windows build, windowed mode): the window can be dragged between 800 x 600
    (smallest) and 1024 x 768 (largest). If it
///     is left outside those limits for a moment, the window snaps back to the nearest allowed size.
    The starting size is 1024 x 768
///     (Player Settings, Resolution and Presentation). Maximize is the exception: pressing the
    maximize button switches to a full
///     screen window at 1920 x 1080 (the screen's own resolution on a 1080p monitor); leaving full
    screen (Alt+Enter) returns to the
///     1024 x 768 window.
///  2. Picture shape: every screen was designed for 16:9. In any other window shape (4:3 for example)
    all cameras draw into the biggest 16:9
///     rectangle that fits and the rest of the window is filled with a dark colour (bars on top and
    bottom, or left and right), so buttons and
///     cards never overlap or fall out of the picture.
/// The limits are the constants below. ClampSize and FitRect are public so the numbers can be checked
    without a window.
/// </summary>
public class ScreenFit : MonoBehaviour
{
    public const int MinWidth = 800;
    public const int MinHeight = 600;
    public const int MaxWidth = 1024;
    public const int MaxHeight = 768;
    private const float ContentAspect = 16f / 9f;
#if !UNITY_EDITOR && UNITY_STANDALONE_WIN
    private const float SnapDelaySeconds = 0.35f;
    private const float FullscreenRequestSeconds = 2f;

    [DllImport("user32.dll")]
    private static extern IntPtr GetActiveWindow();

    [DllImport("user32.dll")]
    private static extern bool IsZoomed(IntPtr window);

    private enum WindowState { Normal, FullscreenRequested, InFullscreen }
#endif
    private static readonly Color BarColor = new Color32(16, 26, 50, 255);

    private static ScreenFit _instance;
    private Camera _barCamera;
    private int _lastWidth;
    private int _lastHeight;
#if !UNITY_EDITOR && UNITY_STANDALONE_WIN
    private float _outsideSince = -1f;
    private WindowState _state = WindowState.Normal;
    private float _requestedAt;
#endif

    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]
    private static void Create()
    {
        if (_instance != null)
        {
            return;
        }
        var go = new GameObject("ScreenFit");
        DontDestroyOnLoad(go);
```

```
    _instance = go.AddComponent<ScreenFit>();
}

private void Awake()
{
    // a camera that only paints the bars: it draws first, clears the whole window and renders
        nothing
    var barGo = new GameObject("BarCamera");
    barGo.transform.SetParent(transform, false);
    _barCamera = barGo.AddComponent<Camera>();
    _barCamera.depth = -100f;
    _barCamera.clearFlags = CameraClearFlags.SolidColor;
    _barCamera.backgroundColor = BarColor;
    _barCamera.cullingMask = 0;
    _barCamera.allowHDR = false;
    _barCamera.allowMSAA = false;
    SceneManager.sceneLoaded += OnSceneLoaded;
}

private void OnDestroy()
{
    SceneManager.sceneLoaded -= OnSceneLoaded;
}

private void OnSceneLoaded(Scene scene, LoadSceneMode mode)
{
    _lastWidth = -1;   // fit the cameras of the new scene on the next frame
}

private void Update()
{
    if (Screen.width != _lastWidth || Screen.height != _lastHeight || _lastWidth == -1)
    {
        _lastWidth = Screen.width;
        _lastHeight = Screen.height;
        ApplyFit();
    }
    LimitWindow();
}

private void LateUpdate()
{
    // cameras that a scene creates in its own Start get the same rectangle
    if (Time.frameCount % 30 == 0)
    {
        ApplyFit();
    }
}

/// <summary>The largest 16:9 rectangle that fits a window of this shape, as a camera viewport (0
    to 1).</summary>
public static Rect FitRect(float windowWidth, float windowHeight)
{
    if (windowWidth <= 0f || windowHeight <= 0f)
    {
        return new Rect(0f, 0f, 1f, 1f);
    }
    float windowAspect = windowWidth / windowHeight;
    if (windowAspect > ContentAspect)
    {
        float width = ContentAspect / windowAspect;      // window wider than 16:9: bars left and
            right
        return new Rect((1f - width) / 2f, 0f, width, 1f);
    }
    float height = windowAspect / ContentAspect;          // window taller than 16:9: bars on top
        and bottom
    return new Rect(0f, (1f - height) / 2f, 1f, height);
}
```

```
    private void ApplyFit()
    {
        Rect rect = FitRect(Screen.width, Screen.height);
        foreach (Camera cam in Camera.allCameras)
        {
            if (cam == _barCamera || cam.targetTexture != null)
            {
                continue;
            }
            if (cam.rect != rect)
            {
                cam.rect = rect;
            }
        }
    }

    /// <summary>The window size limited to 800 x 600 .. 1024 x 768.</summary>
    public static Vector2Int ClampSize(int width, int height)
    {
        return new Vector2Int(Mathf.Clamp(width, MinWidth, MaxWidth), Mathf.Clamp(height, MinHeight,
            MaxHeight));
    }

    /// <summary>
    /// Windows build only. Maximize: switches to a full screen window (1920 x 1080 on a 1080p screen)
        and, when full screen is left again,
    /// returns to the 1024 x 768 window. Otherwise puts a windowed window back inside the allowed
        sizes once it has been outside them for a moment.
    /// </summary>
    private void LimitWindow()
    {
#if !UNITY_EDITOR && UNITY_STANDALONE_WIN
        bool windowed = Screen.fullScreenMode == FullScreenMode.Windowed;
        switch (_state)
        {
            case WindowState.FullscreenRequested:
                if (!windowed)
                {
                    _state = WindowState.InFullscreen;
                }
                else if (Time.unscaledTime - _requestedAt > FullscreenRequestSeconds)
                {
                    _state = WindowState.Normal;
                }
                return;
            case WindowState.InFullscreen:
                if (windowed)
                {
                    // full screen was left: back to the normal window
                    _state = WindowState.Normal;
                    Screen.SetResolution(MaxWidth, MaxHeight, FullScreenMode.Windowed);
                }
                return;
        }

        if (!windowed)
        {
            _outsideSince = -1f;
            return;
        }
        if (IsZoomed(GetActiveWindow()))
        {
            _outsideSince = -1f;
            _state = WindowState.FullscreenRequested;
            _requestedAt = Time.unscaledTime;
            Screen.SetResolution(1920, 1080, FullScreenMode.FullScreenWindow);
            return;
        }
        Vector2Int allowed = ClampSize(Screen.width, Screen.height);
```

```
        if (allowed.x == Screen.width && allowed.y == Screen.height)
        {
            _outsideSince = -1f;
            return;
        }
        if (_outsideSince < 0f)
        {
            _outsideSince = Time.unscaledTime;
        }
        else if (Time.unscaledTime - _outsideSince > SnapDelaySeconds)
        {
            _outsideSince = -1f;
            Screen.SetResolution(allowed.x, allowed.y, FullScreenMode.Windowed);
        }
#endif
    }
}
```

Step 11. LevelDataRegistry: where the level files are

1. Create the script below.
2. Assets > Create > Lakbay Salita > Level Data Registry. Name it **LevelDataRegistry** and move it into `Assets/Main/Resources` (it must be in Resources and have exactly this name).
3. After the JSON files exist (steps 13 to 16), select the asset and drag each language's file into its slot: Wordle Files, Crossword Files, Word Search Files, Phrasebook Files (slot 0 Cebuano, 1 Ilonggo, 2 Tagalog).

Note: the comment in the finished project still names old script folders; the files now live in `Assets/Main/Data/...` as shown in step 3.

Assets/Main/Scripts/Core/LevelDataRegistry.cs

```
using UnityEngine;

/// <summary>
/// LevelDataRegistry
/// Points the games to their per-language level files, so those files can live next to each game's
    scripts instead of in a
/// Resources folder. The one asset, Assets/Main/Resources/LevelDataRegistry.asset, holds three files
    per game, in language order
/// (0 Cebuano, 1 Ilonggo, 2 Tagalog):
///  - Wordle Files:      Assets/Main/Scripts/2 Wordle/Category Levels/&lt;N -
    Language&gt;/wordle_levels_&lt;language&gt;.json
///  - Crossword Files:   Assets/Main/Scripts/3 Crossword/Category Puzzles/&lt;N -
    Language&gt;/crossword_puzzles_&lt;language&gt;.json
///  - Word Search Files: Assets/Main/Scripts/5 Word Search/Category Puzzles/&lt;N -
    Language&gt;/wordsearch_puzzles_&lt;language&gt;.json
/// WordleData, CrosswordCategoryData and WordSearchData read their files through
    LevelDataRegistry.Instance. To swap a language's
/// file, drag another JSON into its slot on the asset in the Inspector.
/// </summary>
[CreateAssetMenu(fileName = "LevelDataRegistry", menuName = "Lakbay Salita/Level Data Registry")]
public class LevelDataRegistry : ScriptableObject
{
    private const string ResourcePath = "LevelDataRegistry";

    [Tooltip("Wordle category levels, one JSON per language: 0 Cebuano, 1 Ilonggo, 2 Tagalog.")]
    public TextAsset[] wordleFiles = new TextAsset[3];

    [Tooltip("Crossword category puzzles, one JSON per language: 0 Cebuano, 1 Ilonggo, 2 Tagalog.")]
    public TextAsset[] crosswordFiles = new TextAsset[3];

    [Tooltip("Word Search category puzzles, one JSON per language: 0 Cebuano, 1 Ilonggo, 2 Tagalog.")]
    public TextAsset[] wordSearchFiles = new TextAsset[3];

    [Tooltip("Phrasebook phrases (Assets/Main/Data/Phrasebook), one JSON per language: 0 Cebuano, 1
        Ilonggo, 2 Tagalog.")]
    public TextAsset[] phrasebookFiles = new TextAsset[3];

    private static LevelDataRegistry _instance;

    /// <summary>The registry asset (Resources/LevelDataRegistry.asset), or null if it is
        missing.</summary>
    public static LevelDataRegistry Instance
    {
        get
        {
            if (_instance == null)
            {
                _instance = Resources.Load<LevelDataRegistry>(ResourcePath);
                if (_instance == null)
                {
                    Debug.LogError("LevelDataRegistry: Resources/" + ResourcePath + ".asset is
                        missing.");
                }
            }
            return _instance;
        }
    }

    /// <summary>
    /// Parses every file of one game (in language order) into T, the per-language data shape. A
        missing or empty slot logs an
    /// error and is returned as null, so the language indexes stay in place.
    /// </summary>
    public static T[] Parse<T>(TextAsset[] files, string gameName) where T : class
    {
        if (files == null)
        {
```

```
            return new T[0];
        }
        var result = new T[files.Length];
        for (int i = 0; i < files.Length; i++)
        {
            if (files[i] == null)
            {
                Debug.LogError("LevelDataRegistry: " + gameName + " file for language " + i + " is not
                    assigned.");
                continue;
            }
            result[i] = JsonUtility.FromJson<T>(files[i].text);
        }
        return result;
    }
}
```

Part 4. Data (JSON)

All content is JSON read with Unity's `JsonUtility`. Rules that apply to every file: UTF-8, field names exactly as shown, every list non-empty, language order Cebuano, Ilonggo, Tagalog, and the top-level fields `purpose` and `status` are free text for humans (they are ignored by the game).

Step 12. The dictionary files (Library)

Create `Assets/Main/Data/Language Dictionary/cebuano.json`, `hiligaynon.json` and `tagalog.json`. Each is a JSON **array** of entries. The loader (`LibraryData`) also accepts a wrapper object with an `items` array.

Assets/Main/Data/Language Dictionary/cebuano.json (first entries)

```
[
  {
    "word": "ABA",
    "translation": "exclamation of pleasure and surprise",
    "definition": "Exclamation of pleasure and surprise; breast of fowls.",
    "letters": 3,
    "language": "Cebuano"
  },
  {
    "word": "ABI",
    "translation": "— thought, took for granted",
    "definition": "— thought, took for granted.",
    "letters": 3,
    "language": "Cebuano"
  },
  {
    "word": "ABO",
    "translation": "ash",
    "definition": "The gray powder left behind after something burns; also the color gray.",
    "letters": 3,
    "language": "Cebuano"
  },
  {
    "word": "ABU",
    "translation": "ashes",
    "definition": "Ashes; see alabúhan; v. make ashes.",
    "letters": 3,
    "language": "Cebuano"
  },
  {
    "word": "ADA",
    "tra
  ...
]
```

Field	Meaning
word	The native word in capital letters, letters A to Z
translation	A short English meaning (used for search and clues)
definition	A longer English definition
letters	Number of letters of `word`
language	"Cebuano", "Hiligaynon" or "Tagalog"

- The finished project has **22,005** Cebuano, **15,223** Hiligaynon and **26,312** Tagalog entries (63,540 in total).

- Some entries start with a dash or contain a stray replacement character; `LibraryData` cleans these when it loads. An entry with an empty meaning is kept but marked.
- Source the words from a dictionary you are allowed to use and convert them to this format with a script.

Step 13. Wordle levels

One file per language: `Assets/Main/Data/Wordle/Levels/wordle_levels_<language>.json` (`cebuano`, `ilonggo`, `tagalog`). Seven categories with six levels each; words are 4 to 7 letters and get longer with each level; letters A to Z only.

Assets/Main/Data/Wordle/Levels/wordle_levels_cebuano.json

```
{
 "purpose": "Wordle levels for Lakbay Salita, Cebuano only: 7 categories x 6 levels. Each level is one
     word (4-7 letters, growing with the level) with its English meaning and a dictionary clue. Loaded
     at runtime by WordleData through Resources/LevelDataRegistry.asset (Wordle Files, slot 0).",
 "status": "DRAFT - the words were picked automatically from the dictionary files (clean single-meaning
     entries first, one meaning per level) and still need a native-speaker review. Edit this file to
     change a word; keep each category's levels in order from shortest to longest word. Words must use
     letters A-Z only.",
 "name": "Cebuano",
 "categories": [
  {
   "name": "Animals",
   "levels": [
    {
     "word": "ISDA",
     "meaning": "fish",
     "clue": "Fish, whether alive or as food."
    },
    {
     "word": "ITIK",
     "meaning": "duck",
     "clue": "Duck."
    },
    {
     "word": "BAKI",
     "meaning": "frog",
     "clue": "Frog; toad."
    },
    {
     "word": "MANOK",
     "meaning": "chicken",
     "clue": "A chicken, rooster, or fowl."
    },
    {
     "word": "KABAYO",
     "meaning": "horse",
     "clue": "A horse."
    },
    {
     "word": "LANGGAM",
     "meaning": "bird",
     "clue": "Bird; fowl; swallow."
    }
  ...
```

The C# data model and the loader. The file is parsed once through the registry and cached:

Assets/Main/Scripts/2 Wordle/WordleData.cs

```
using System;
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// WordleData
/// The data model and loader for the category levels of Wordle. The levels live in one JSON file per
    language,
/// Assets/Main/Data/Wordle/Levels/wordle_levels_<language>.json, found through LevelDataRegistry: 3
    languages (0 Cebuano, 1 Ilonggo, 2 Tagalog) x 7 categories x 6 levels. A level is one
/// word (4-7 letters, longer with every level) with its English meaning and a dictionary clue.
    WordleData.Level(language, category, level)
/// returns one level; the file is read once and cached. To change a word, edit the JSON (letters A-Z
    only, categories in the same order
/// as the Word Search categories). CategoryColors holds the colour dot of each category, shared by the
    start screen and the play screen.
/// </summary>
[Serializable]
public class WordleLevelDef
{
    public string word;
    public string meaning;
    public string clue;
}

[Serializable]
public class WordleCategoryDef
{
    public string name;
    public List<WordleLevelDef> levels = new List<WordleLevelDef>();
}

[Serializable]
public class WordleLanguageDef
{
    public string name;
    public List<WordleCategoryDef> categories = new List<WordleCategoryDef>();
}

[Serializable]
public class WordleLevelsFile
{
    public string purpose;
    public string status;
    public List<WordleLanguageDef> languages = new List<WordleLanguageDef>();
}

public static class WordleData
{
    private static WordleLevelsFile _file;

    /// <summary>Language names in the order of the language index (0 Cebuano, 1 Ilonggo, 2
        Tagalog).</summary>
    public static readonly string[] LanguageNames = { "Cebuano", "Ilonggo", "Tagalog" };

    /// <summary>The colour of each category's dot (Animals, Food and drink, Body, Family and people,
        Home and things, Nature and weather, Numbers).</summary>
    public static readonly Color[] CategoryColors =
    {
        new Color32(217, 96, 59, 255), new Color32(232, 185, 35, 255), new Color32(93, 160, 42, 255),
            new Color32(138, 63, 199, 255),
        new Color32(55, 138, 221, 255), new Color32(29, 158, 117, 255), new Color32(232, 117, 26, 255)
    };

    /// <summary>The whole level file (null if it is missing).</summary>
    public static WordleLevelsFile Data
    {
```

```
    get
    {
        if (_file == null)
        {
            LevelDataRegistry registry = LevelDataRegistry.Instance;
            if (registry == null)
            {
                return null;
            }
            // one JSON per language, in language order (see LevelDataRegistry)
            _file = new WordleLevelsFile();
            _file.languages.AddRange(LevelDataRegistry.Parse<WordleLanguageDef>(registry.wordleFiles
                , "Wordle"));
        }
        return _file;
    }
}

/// <summary>Forgets the cached file so the next call reads the JSON again.</summary>
public static void Reload()
{
    _file = null;
}

public static WordleLanguageDef Language(int language)
{
    WordleLevelsFile file = Data;
    if (file == null || language < 0 || language >= file.languages.Count)
    {
        return null;
    }
    return file.languages[language];
}

public static int CategoryCount(int language)
{
    WordleLanguageDef l = Language(language);
    return l != null ? l.categories.Count : 0;
}

public static WordleCategoryDef Category(int language, int category)
{
    WordleLanguageDef l = Language(language);
    if (l == null || category < 0 || category >= l.categories.Count)
    {
        return null;
    }
    return l.categories[category];
}

public static int LevelCount(int language, int category)
{
    WordleCategoryDef c = Category(language, category);
    return c != null ? c.levels.Count : 0;
}

public static WordleLevelDef Level(int language, int category, int level)
{
    WordleCategoryDef c = Category(language, category);
    if (c == null || level < 0 || level >= c.levels.Count)
    {
        return null;
    }
    return c.levels[level];
}

/// <summary>The colour dot of a category (wraps around if there are more categories than
    colours).</summary>
public static Color CategoryColor(int category)
```

```
    {
        return CategoryColors[Mathf.Abs(category) % CategoryColors.Length];
    }

    /// <summary>How many levels a language has in total (all categories).</summary>
    public static int TotalLevels(int language)
    {
        int total = 0;
        for (int c = 0; c < CategoryCount(language); c++)
        {
            total += LevelCount(language, c);
        }
        return total;
    }
}
```

`WordleSession` hands the chosen language, category and level to the play scene and finds the next level; `WordleProgressStore` saves solved levels:

Assets/Main/Scripts/2 Wordle/WordleSession.cs

```
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// WordleSession
/// Static hand-over between the Wordle category scenes: which language, category and level (0-based)
    to play. The language hub sets the
/// language, the start screen sets the category and level, and the play scene ("Wordle - Play") reads
    them. Language index everywhere:
/// 0 Cebuano, 1 Ilonggo, 2 Tagalog. Also holds the scene names and finds the level that comes after
    the current one.
/// </summary>
public static class WordleSession
{
    public const string StartScene = "Wordle - Start";
    public const string PlayScene = "Wordle - Play";

    public static int Language;
    public static int Category;
    public static int Level;

    /// <summary>The level after (category, level): the next level of the category, then the first
        level of the next category. False after the very last one.</summary>
    public static bool TryGetNext(int language, int category, int level, out int nextCategory, out int
        nextLevel)
    {
        nextCategory = category;
        nextLevel = level + 1;
        if (nextLevel < WordleData.LevelCount(language, category))
        {
            return true;
        }
        nextCategory = category + 1;
        nextLevel = 0;
        return nextCategory < WordleData.CategoryCount(language);
    }
}

/// <summary>
/// WordleProgressStore
/// Remembers which Wordle category levels the player has solved, in PlayerPrefs ("wordle.solved.v1",
    entries "language:category:level"
/// joined with "|"). The start screen shows "N of 6 solved" per category from it, and the hub shows
    the total.
/// </summary>
public static class WordleProgressStore
{
    private const string SolvedKey = "wordle.solved.v1";

    private static string KeyFor(int language, int category, int level)
    {
        return language + ":" + category + ":" + level;
    }

    private static List<string> Load()
    {
        var list = new List<string>();
        string saved = PlayerPrefs.GetString(SolvedKey, string.Empty);
        foreach (string part in saved.Split('|'))
        {
            if (part.Length > 0)
            {
                list.Add(part);
            }
        }
        return list;
    }
```

```
    public static bool IsSolved(int language, int category, int level)
    {
        return Load().Contains(KeyFor(language, category, level));
    }

    /// <summary>How many of a category's levels are solved.</summary>
    public static int SolvedCount(int language, int category, int levelCount)
    {
        List<string> list = Load();
        int count = 0;
        for (int l = 0; l < levelCount; l++)
        {
            if (list.Contains(KeyFor(language, category, l)))
            {
                count++;
            }
        }
        return count;
    }

    /// <summary>How many levels are solved in one language (all categories).</summary>
    public static int SolvedTotal(int language)
    {
        int total = 0;
        for (int c = 0; c < WordleData.CategoryCount(language); c++)
        {
            total += SolvedCount(language, c, WordleData.LevelCount(language, c));
        }
        return total;
    }

    public static void MarkSolved(int language, int category, int level)
    {
        List<string> list = Load();
        string key = KeyFor(language, category, level);
        if (!list.Contains(key))
        {
            list.Add(key);
            PlayerPrefs.SetString(SolvedKey, string.Join("|", list.ToArray()));
            PlayerPrefs.Save();
        }
    }

    /// <summary>Forgets every solved level (used when the profile is reset).</summary>
    public static void Reset()
    {
        PlayerPrefs.DeleteKey(SolvedKey);
        PlayerPrefs.Save();
    }
}
```

Step 14. Crossword puzzles

`crossword_puzzles_<language>.json`: seven categories, each with three puzzles (Small 5 words, Medium 8, Large 11). A word has its answer, clue, grid position (`row`, `col` from the top-left), direction (`across` true or false) and clue `number`. **The grids are produced by a generator, not typed by hand.** Use the generator scripts in `Scripts/3 Crossword/Level Loader` (`CrosswordPuzzleGenerator`, `CrosswordLevelLoader`; they need Newtonsoft Json) to lay out words so that every crossing letter matches, then write the result to this format.

Assets/Main/Data/Crossword/Puzzles/crossword_puzzles_cebuano.json


```
{
 "purpose": "Crossword puzzles for Lakbay Salita, Cebuano only: 7 categories x 3 sizes (Small 5 words,
     Medium 8, Large 11). Each word has its grid position (row and col from the top-left, across = true
     for across and false for down), its clue number and its clue. Loaded at runtime by
     CrosswordCategoryData through Resources/LevelDataRegistry.asset (Crossword Files, slot 0).",
 "status": "DRAFT - the words were picked automatically from the dictionary files (clean single-meaning
     entries first, one meaning per word) and the grids were laid out by a generator. The words still
     need a native-speaker review. To change a word the grid has to be generated again, so send the
     corrections and ask for the puzzles to be rebuilt instead of editing the positions by hand.",
 "name": "Cebuano",
 "categories": [
  {
   "name": "Animals",
   "puzzles": [
    {
     "size": "Small",
     "words": [
      {
       "answer": "IRO",
       "clue": "This word means a dog.",
       "row": 0,
       "col": 1,
       "across": true,
       "number": 1
      },
      {
       "answer": "ISDA",
       "clue": "Fish, whether alive or as food.",
       "row": 0,
       "col": 1,
       "across": false,
       "number": 1
      },
      {
       "answer": "MANOK",
       "clue": "A chicken, rooster, or fowl.",
       "row": 2,
       "col": 5,
       "across": false,
       "number": 2
      },
      {
       "answer": "LANGGAM",
       "clue": "This word means bird; fowl; swallow.",
       "row": 3,
       "col": 0,
       "across": true,
       "number": 3
      },
      {
       "answer": "ITIK",
       "clue": "This word means duck.",
  ...
```


How the generator avoids broken puzzles: it records which direction (Across or Down) owns each cell, and lets a new word reuse a cell only when that cell belongs to the **opposite** direction, so words truly cross instead of overlapping.

The in-game puzzle model (`CrosswordPuzzle`, words with their cells).

Assets/Main/Scripts/3 Crossword/Gameplay/CrosswordPuzzleData.cs

```
using System.Collections.Generic;

/// <summary>
/// CrosswordPuzzleData
/// Shared data types for the hardcoded per-level crossword puzzles in Assets/Main/Data
/// (e.g. "cebuano - level 1.cs" through "tagalog - level 10.cs"). Each puzzle file defines
/// one static CrosswordPuzzle listing its Across/Down words with a grid position (0-indexed,
/// row 0 = top, col 0 = left) - any cell not covered by a word's letters is a black/blocked
/// cell, meant to be derived at runtime by whatever generator consumes this data rather than
/// being stored explicitly here. "number" follows standard crossword numbering (top-left to
/// bottom-right scan; a cell is numbered once if it starts an Across and/or Down entry).
/// </summary>
public enum CrosswordDirection
{
    Across,
    Down
}

[System.Serializable]
public class CrosswordWordEntry
{
    public int number;
    public string answer;
    public string clue;
    public int row;
    public int col;
    public CrosswordDirection direction;

    public CrosswordWordEntry(int number, string answer, string clue, int row, int col,
        CrosswordDirection direction)
    {
        this.number = number;
        this.answer = answer;
        this.clue = clue;
        this.row = row;
        this.col = col;
        this.direction = direction;
    }
}

public class CrosswordPuzzle
{
    public string language;
    public int level;
    public List<CrosswordWordEntry> words;

    public CrosswordPuzzle(string language, int level, List<CrosswordWordEntry> words)
    {
        this.language = language;
        this.level = level;
        this.words = words;
    }
}
```

The loader for the JSON; `ToPuzzle` converts a JSON puzzle into the in-game model.

Assets/Main/Scripts/3 Crossword/CrosswordCategoryData.cs

```
using System;
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// CrosswordCategoryData
/// The data model and loader for the category crosswords. The puzzles live in one JSON file per
    language,
/// Assets/Main/Data/Crossword/Puzzles/crossword_puzzles_<language>.json, found through
    LevelDataRegistry: 3 languages (0 Cebuano, 1 Ilonggo, 2 Tagalog) x 7 categories x 3 sizes (Small 5
    words, Medium 8, Large 11). Every
/// answer of a puzzle belongs to its category. A word has its grid position (row and col counted from
    the top-left, across = true for an
/// across word and false for a down word), its clue number and its clue.
    CrosswordCategoryData.Puzzle(...) returns a puzzle definition and
/// ToPuzzle turns it into the CrosswordPuzzle the game already plays
    (CrosswordInputController.Initialize). The JSON is read once and
/// cached. The grids were generated, so to change a word ask for the puzzles to be rebuilt instead of
    moving letters by hand.
/// </summary>
[Serializable]
public class CrosswordWordDef
{
    public string answer;
    public string clue;
    public int row;
    public int col;
    public bool across;
    public int number;
}

[Serializable]
public class CrosswordPuzzleDef
{
    public string size;
    public List<CrosswordWordDef> words = new List<CrosswordWordDef>();
}

[Serializable]
public class CrosswordCategoryDef
{
    public string name;
    public List<CrosswordPuzzleDef> puzzles = new List<CrosswordPuzzleDef>();
}

[Serializable]
public class CrosswordLanguageDef
{
    public string name;
    public List<CrosswordCategoryDef> categories = new List<CrosswordCategoryDef>();
}

[Serializable]
public class CrosswordCategoriesFile
{
    public string purpose;
    public string status;
    public List<CrosswordLanguageDef> languages = new List<CrosswordLanguageDef>();
}

public static class CrosswordCategoryData
{
    private static CrosswordCategoriesFile _file;

    /// <summary>Language names in the order of the language index (0 Cebuano, 1 Ilonggo, 2
        Tagalog).</summary>
    public static readonly string[] LanguageNames = { "Cebuano", "Ilonggo", "Tagalog" };
```

```
/// <summary>The whole puzzle file (null if it is missing).</summary>
public static CrosswordCategoriesFile Data
{
    get
    {
        if (_file == null)
        {
            LevelDataRegistry registry = LevelDataRegistry.Instance;
            if (registry == null)
            {
                return null;
            }
            // one JSON per language, in language order (see LevelDataRegistry)
            _file = new CrosswordCategoriesFile();
            _file.languages.AddRange(LevelDataRegistry.Parse<CrosswordLanguageDef>(registry.crosswor
                dFiles, "Crossword"));
        }
        return _file;
    }
}

/// <summary>Forgets the cached file so the next call reads the JSON again.</summary>
public static void Reload()
{
    _file = null;
}

public static CrosswordLanguageDef Language(int language)
{
    CrosswordCategoriesFile file = Data;
    if (file == null || language < 0 || language >= file.languages.Count)
    {
        return null;
    }
    return file.languages[language];
}

public static int CategoryCount(int language)
{
    CrosswordLanguageDef l = Language(language);
    return l != null ? l.categories.Count : 0;
}

public static CrosswordCategoryDef Category(int language, int category)
{
    CrosswordLanguageDef l = Language(language);
    if (l == null || category < 0 || category >= l.categories.Count)
    {
        return null;
    }
    return l.categories[category];
}

public static int SizeCount(int language, int category)
{
    CrosswordCategoryDef c = Category(language, category);
    return c != null ? c.puzzles.Count : 0;
}

public static CrosswordPuzzleDef Puzzle(int language, int category, int size)
{
    CrosswordCategoryDef c = Category(language, category);
    if (c == null || size < 0 || size >= c.puzzles.Count)
    {
        return null;
    }
    return c.puzzles[size];
}
```

```
    /// <summary>How many puzzles a language has in total (all categories and sizes).</summary>
    public static int TotalPuzzles(int language)
    {
        int total = 0;
        for (int c = 0; c < CategoryCount(language); c++)
        {
            total += SizeCount(language, c);
        }
        return total;
    }

    /// <summary>Turns a puzzle definition into the CrosswordPuzzle the crossword game plays. The
        "level" is the size number (1 Small, 2 Medium, 3 Large).</summary>
    public static CrosswordPuzzle ToPuzzle(int language, int size, CrosswordPuzzleDef def)
    {
        var words = new List<CrosswordWordEntry>();
        foreach (CrosswordWordDef w in def.words)
        {
            words.Add(new CrosswordWordEntry(w.number, w.answer, w.clue, w.row, w.col,
                w.across ? CrosswordDirection.Across : CrosswordDirection.Down));
        }
        return new CrosswordPuzzle(LanguageNames[Mathf.Clamp(language, 0, 2)], size + 1, words);
    }
}
```

Step 15. Word Search puzzles

`wordsearch_puzzles_<language>.json`: seven categories, each with three puzzles (ids A, B, C) of 7 to 8 words of 3 to 8 letters, each with its English meaning. The grid itself is **not stored**: `WordSearchGenerator` hides the words in a 10 x 10 grid with a fixed seed.

Assets/Main/Data/Word Search/Puzzles/wordsearch_puzzles_cebuano.json


```
{
 "purpose": "Word Search puzzles for Lakbay Salita, Cebuano only: 7 categories x 3 puzzles (A, B, C) of
     7-8 native words, each with its English meaning. Words are 3-8 letters so they fit the 10x10 grid.
     Loaded at runtime by WordSearchData through Resources/LevelDataRegistry.asset (Word Search Files,
     slot 0).",
 "status": "DRAFT - the words were picked automatically from the dictionary files and still need a
     native-speaker review (spelling, dialect, loanwords such as the Spanish-style numbers). Edit this
     file to change a word; keep each puzzle at 7-8 words.",
 "name": "Cebuano",
 "categories": [
  {
   "name": "Animals",
   "puzzles": [
    {
     "id": "A",
     "words": [
      {
       "word": "IRO",
       "meaning": "dog"
      },
      {
       "word": "MANOK",
       "meaning": "chicken"
      },
      {
       "word": "LANGGAM",
       "meaning": "bird"
      },
      {
       "word": "ISDA",
       "meaning": "fish"
      },
      {
       "word": "ITIK",
       "meaning": "duck"
      },
      {
       "word": "BITIN",
       "meaning": "snake"
  ...
```

Assets/Main/Scripts/4 Word Search/WordSearchData.cs

```
using System;
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// WordSearchData
/// The Word Search puzzle data model and its loader. The puzzles live in one JSON file per language,
    Assets/Main/Data/Word Search/Puzzles/
/// wordsearch_puzzles_<language>.json, found through LevelDataRegistry: 3 languages (0 Cebuano, 1
    Ilonggo, 2 Tagalog) x 7 categories x 3 puzzles (A, B, C) of 7-8 words,
/// each word with its English meaning. WordSearchData.Puzzle(language, category, puzzle) returns one
    puzzle; the file is read
/// once and cached. To change a word, edit the JSON (keep words 3-8 letters so they fit the 10x10
    grid).
/// </summary>
[Serializable]
public class WordSearchWord
{
    public string word;
    public string meaning;
}

[Serializable]
public class WordSearchPuzzleDef
{
    public string id;
    public List<WordSearchWord> words = new List<WordSearchWord>();
}

[Serializable]
public class WordSearchCategory
{
    public string name;
    public List<WordSearchPuzzleDef> puzzles = new List<WordSearchPuzzleDef>();
}

[Serializable]
public class WordSearchLanguage
{
    public string name;
    public List<WordSearchCategory> categories = new List<WordSearchCategory>();
}

[Serializable]
public class WordSearchFile
{
    public string purpose;
    public string status;
    public List<WordSearchLanguage> languages = new List<WordSearchLanguage>();
}

public static class WordSearchData
{
    private static WordSearchFile _file;

    /// <summary>Language names in the order of the language index (0 Cebuano, 1 Ilonggo, 2
        Tagalog).</summary>
    public static readonly string[] LanguageNames = { "Cebuano", "Ilonggo", "Tagalog" };

    /// <summary>The whole puzzle file (null if it is missing).</summary>
    public static WordSearchFile Data
    {
        get
        {
            if (_file == null)
            {
                LevelDataRegistry registry = LevelDataRegistry.Instance;
```

```
                if (registry == null)
                {
                    return null;
                }
                // one JSON per language, in language order (see LevelDataRegistry)
                _file = new WordSearchFile();
                _file.languages.AddRange(LevelDataRegistry.Parse<WordSearchLanguage>(registry.wordSearch
                    Files, "Word Search"));
            }
            return _file;
        }
    }

    /// <summary>Forgets the cached file so the next call reads the JSON again (used after editing it
        in the editor).</summary>
    public static void Reload()
    {
        _file = null;
    }

    public static WordSearchLanguage Language(int language)
    {
        WordSearchFile file = Data;
        if (file == null || language < 0 || language >= file.languages.Count)
        {
            return null;
        }
        return file.languages[language];
    }

    public static int CategoryCount(int language)
    {
        WordSearchLanguage l = Language(language);
        return l != null ? l.categories.Count : 0;
    }

    public static WordSearchCategory Category(int language, int category)
    {
        WordSearchLanguage l = Language(language);
        if (l == null || category < 0 || category >= l.categories.Count)
        {
            return null;
        }
        return l.categories[category];
    }

    /// <summary>How many puzzles a language has in total (all categories).</summary>
    public static int TotalPuzzles(int language)
    {
        int total = 0;
        for (int c = 0; c < CategoryCount(language); c++)
        {
            WordSearchCategory category = Category(language, c);
            total += category != null ? category.puzzles.Count : 0;
        }
        return total;
    }

    public static WordSearchPuzzleDef Puzzle(int language, int category, int puzzle)
    {
        WordSearchCategory c = Category(language, category);
        if (c == null || puzzle < 0 || puzzle >= c.puzzles.Count)
        {
            return null;
        }
        return c.puzzles[puzzle];
    }
}
```

Words run across, down or diagonally down-right (never backwards), may cross where letters match, and the rest of the grid is filled with random letters taken mostly from the puzzle's own words.

Assets/Main/Scripts/4 Word Search/WordSearchGenerator.cs

```
using System;
using System.Collections.Generic;

/// <summary>Where one word sits in the grid: start cell, direction (dRow/dCol) and the word with its
    meaning.</summary>
public class WordSearchPlacement
{
    public string word;
    public string meaning;
    public int row;
    public int col;
    public int dRow;
    public int dCol;
    public bool found;
    public int colorIndex = -1;

    public int EndRow { get { return row + dRow * (word.Length - 1); } }
    public int EndCol { get { return col + dCol * (word.Length - 1); } }
}

/// <summary>A generated puzzle: the 10x10 letters and where each word was hidden.</summary>
public class WordSearchGrid
{
    public const int Size = 10;
    public char[,] letters = new char[Size, Size];
    public List<WordSearchPlacement> placements = new List<WordSearchPlacement>();
}

/// <summary>
/// WordSearchGenerator
/// Hides a puzzle's words in a 10x10 grid. Words run across, down or diagonally down-right (never
    backwards); words may cross where
/// their letters match. The remaining cells are filled with random letters, mostly taken from the
    puzzle's own words so the grid
/// looks natural. The same seed always gives the same grid, so every puzzle keeps one fixed layout.
/// </summary>
public static class WordSearchGenerator
{
    private static readonly int[,] Directions = { { 0, 1 }, { 1, 0 }, { 1, 1 } };
    private const string Alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";

    public static WordSearchGrid Generate(IList<WordSearchWord> words, int seed)
    {
        var ordered = new List<WordSearchWord>(words);
        ordered.Sort((a, b) => b.word.Length.CompareTo(a.word.Length));

        for (int attempt = 0; attempt < 400; attempt++)
        {
            var rng = new Random(seed + attempt * 101);
            WordSearchGrid grid = TryPlace(ordered, rng);
            if (grid != null)
            {
                Fill(grid, ordered, rng);
                return grid;
            }
        }
        return null;
    }

    private static WordSearchGrid TryPlace(List<WordSearchWord> ordered, Random rng)
    {
        var grid = new WordSearchGrid();
        for (int r = 0; r < WordSearchGrid.Size; r++)
        {
            for (int c = 0; c < WordSearchGrid.Size; c++)
            {
                grid.letters[r, c] = '\0';
```

```
            }
        }

        foreach (WordSearchWord w in ordered)
        {
            string text = w.word.Trim().ToUpperInvariant();
            bool placed = false;
            for (int tries = 0; tries < 300 && !placed; tries++)
            {
                int d = rng.Next(Directions.GetLength(0));
                int dr = Directions[d, 0];
                int dc = Directions[d, 1];
                int r = rng.Next(WordSearchGrid.Size);
                int c = rng.Next(WordSearchGrid.Size);
                if (!Fits(grid, text, r, c, dr, dc))
                {
                    continue;
                }
                for (int i = 0; i < text.Length; i++)
                {
                    grid.letters[r + dr * i, c + dc * i] = text[i];
                }
                grid.placements.Add(new WordSearchPlacement { word = text, meaning = w.meaning, row =
                    r, col = c, dRow = dr, dCol = dc });
                placed = true;
            }
            if (!placed)
            {
                return null;
            }
        }
        return grid;
    }

    private static bool Fits(WordSearchGrid grid, string text, int r, int c, int dr, int dc)
    {
        int endR = r + dr * (text.Length - 1);
        int endC = c + dc * (text.Length - 1);
        if (endR >= WordSearchGrid.Size || endC >= WordSearchGrid.Size)
        {
            return false;
        }
        bool crossesAnother = false;
        for (int i = 0; i < text.Length; i++)
        {
            char existing = grid.letters[r + dr * i, c + dc * i];
            if (existing != '\0')
            {
                if (existing != text[i])
                {
                    return false;
                }
                crossesAnother = true;
            }
        }
        // a word may not sit completely on top of letters that already belong to other words
        return !(crossesAnother && AllCellsFilled(grid, text, r, c, dr, dc));
    }

    private static bool AllCellsFilled(WordSearchGrid grid, string text, int r, int c, int dr, int dc)
    {
        for (int i = 0; i < text.Length; i++)
        {
            if (grid.letters[r + dr * i, c + dc * i] == '\0')
            {
                return false;
            }
        }
        return true;
```

```
    }

    private static void Fill(WordSearchGrid grid, List<WordSearchWord> words, Random rng)
    {
        var pool = new List<char>();
        foreach (WordSearchWord w in words)
        {
            foreach (char ch in w.word.ToUpperInvariant())
            {
                if (Alphabet.IndexOf(ch) >= 0)
                {
                    pool.Add(ch);
                }
            }
        }
        for (int r = 0; r < WordSearchGrid.Size; r++)
        {
            for (int c = 0; c < WordSearchGrid.Size; c++)
            {
                if (grid.letters[r, c] == '\0')
                {
                    bool fromWords = pool.Count > 0 && rng.Next(100) < 60;
                    grid.letters[r, c] = fromWords ? pool[rng.Next(pool.Count)] :
                        Alphabet[rng.Next(Alphabet.Length)];
                }
            }
        }
    }
}
```

Step 16. Phrasebook phrases

`Assets/Main/Data/Phrasebook/phrases_<language>.json`: seven topics (Greetings, Thanks and sorry, About me, Food and market, Directions, Numbers and time, Help and safety). A phrase has the English line, the phrase in the language and an optional `gloss` list: **empty, or exactly one entry per word** (words are split at spaces).

Assets/Main/Data/Phrasebook/phrases_cebuano.json


```
{
 "purpose": "Phrasebook phrases for Lakbay Salita, Cebuano only: 7 everyday topics, each phrase with
     its English line, the phrase in the language and optional word-by-word meanings (gloss, one entry
     per word, same order). The Phrasebook screens and the Sentence builder split a phrase into words
     at the spaces. Loaded at runtime by PhraseData through Resources/LevelDataRegistry.asset
     (Phrasebook Files).",
 "status": "DRAFT PLACEHOLDERS - these few phrases only test the Phrasebook screens. Every phrase must
     be checked or replaced by a native speaker before the game is shared (Ilonggo especially). Edit
     this file to change a phrase; keep gloss empty or exactly one entry per word.",
 "name": "Cebuano",
 "topics": [
  {
   "name": "Greetings",
   "phrases": [
    {
     "english": "Good morning",
     "phrase": "Maayong buntag",
     "gloss": [
      "good",
      "morning"
     ]
    },
    {
     "english": "Good afternoon",
     "phrase": "Maayong hapon",
     "gloss": [
      "good",
      "afternoon"
     ]
    },
    {
     "english": "Good evening",
     "phrase": "Maayong gabii",
     "gloss": [
      "good",
      "evening"
     ]
    },
    {
     "english": "How are you?",
     "phrase": "Kumusta ka?",
     "gloss": [
      "how are",
      "you"
     ]
    },
    {
     "english": "I am fine",
     "phrase": "Maayo ra ko",
     "gloss": []
  ...
```

Assets/Main/Scripts/7 Phrasebook/PhraseData.cs

```
using System;
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// PhraseData
/// The Phrasebook's data model and loader. The phrases live in one JSON file per language
    (Assets/Main/Data/Phrasebook/phrases_&lt;language&gt;.json),
/// found through LevelDataRegistry (Phrasebook Files): 3 languages (0 Cebuano, 1 Ilonggo, 2 Tagalog) x
    7 topics x a few phrases. A phrase has the
/// English line, the phrase in the language and optional word-by-word meanings (gloss: empty, or one
    entry per word). The Phrasebook screens and
/// the Sentence builder split a phrase into words at its spaces (Tokens). The file is read once and
    cached. The phrases are placeholders until
/// native speakers check them.
/// </summary>
[Serializable]
public class PhraseEntry
{
    public string english;
    public string phrase;
    public List<string> gloss = new List<string>();
}

[Serializable]
public class PhraseTopicDef
{
    public string name;
    public List<PhraseEntry> phrases = new List<PhraseEntry>();
}

[Serializable]
public class PhraseLanguageDef
{
    public string name;
    public List<PhraseTopicDef> topics = new List<PhraseTopicDef>();
}

public static class PhraseData
{
    private static PhraseLanguageDef[] _languages;

    /// <summary>Language names in the order of the language index (0 Cebuano, 1 Ilonggo, 2
        Tagalog).</summary>
    public static readonly string[] LanguageNames = { "Cebuano", "Ilonggo", "Tagalog" };

    /// <summary>Forgets the cached file so the next call reads the JSON again.</summary>
    public static void Reload()
    {
        _languages = null;
    }

    public static PhraseLanguageDef Language(int language)
    {
        if (_languages == null)
        {
            LevelDataRegistry registry = LevelDataRegistry.Instance;
            if (registry == null)
            {
                return null;
            }
            _languages = LevelDataRegistry.Parse<PhraseLanguageDef>(registry.phrasebookFiles,
                "Phrasebook");
        }
        return language >= 0 && language < _languages.Length ? _languages[language] : null;
    }
```

```
    public static int TopicCount(int language)
    {
        PhraseLanguageDef l = Language(language);
        return l != null ? l.topics.Count : 0;
    }

    public static PhraseTopicDef Topic(int language, int topic)
    {
        PhraseLanguageDef l = Language(language);
        return l != null && topic >= 0 && topic < l.topics.Count ? l.topics[topic] : null;
    }

    public static int PhraseCount(int language, int topic)
    {
        PhraseTopicDef t = Topic(language, topic);
        return t != null ? t.phrases.Count : 0;
    }

    public static PhraseEntry Phrase(int language, int topic, int phrase)
    {
        PhraseTopicDef t = Topic(language, topic);
        return t != null && phrase >= 0 && phrase < t.phrases.Count ? t.phrases[phrase] : null;
    }

    /// <summary>How many phrases a language has in total (all topics).</summary>
    public static int TotalPhrases(int language)
    {
        int total = 0;
        for (int t = 0; t < TopicCount(language); t++)
        {
            total += PhraseCount(language, t);
        }
        return total;
    }

    /// <summary>The words of a phrase, split at the spaces; punctuation stays on the word it
        follows.</summary>
    public static string[] Tokens(string phrase)
    {
        if (string.IsNullOrEmpty(phrase))
        {
            return new string[0];
        }
        return phrase.Trim().Split(new[] { ' ' }, StringSplitOptions.RemoveEmptyEntries);
    }
}
```

Assets/Main/Scripts/7 Phrasebook/PhraseMenuButton.cs

```
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;

/// <summary>
/// PhraseMenuButton
/// Put on the Phrasebook card of the main menu (1 Main_menu), next to a Button on the same object.
    Pressing it opens the Phrasebook topics screen
/// ("Phrasebook - Topics"), in the language the player picked there last time (Cebuano the first
    time). The language is chosen on the topics
/// screen itself, like the tabs of the Library.
/// </summary>
[RequireComponent(typeof(Button))]
public class PhraseMenuButton : MonoBehaviour
{
    private void Awake()
    {
        GetComponent<Button>().onClick.AddListener(Open);
    }

    /// <summary>Opens the Phrasebook.</summary>
    public void Open()
    {
        PhraseSession.Language = PhraseSession.SavedLanguage();
        SceneManager.LoadScene(PhraseSession.TopicsScene);
    }
}
```

`PhraseSession` (hand-over of language, topic and phrase, and the `PhraseProgressStore` of learned phrases) is in `Scripts/7 Phrasebook/PhraseSession.cs`; it follows the same pattern as `WordleSession`.

Putting the files in the registry

Now open `Resources/LevelDataRegistry.asset` and fill the slots (step 11): Wordle Files, Crossword Files, Word Search Files and Phrasebook Files, three each, in language order. A missing file logs an error naming the game and the language.

An example data script (not part of the project)

The project's own data scripts are not in the repository. This small example shows the idea of turning a dictionary into a Wordle file: take clean single-meaning words, group them by topic, order each topic from short to long.

make_wordle_json.py (example)

```
import json
# topics[name] = list of (word, meaning, clue) already chosen from the dictionary
def make_level_file(language, topics):
    cats = []
    for name, words in topics.items():
        words = sorted(words, key=lambda w: len(w[0]))[:6]          # 6 levels, short to long
        assert all(w[0].isalpha() and w[0].isupper() and 4 <= len(w[0]) <= 7 for w in words)
        cats.append({"name": name, "levels": [{"word": w, "meaning": m, "clue": c} for w, m, c in
            words]})
    return {"purpose": "Wordle levels", "status": "DRAFT", "name": language, "categories": cats}

with open("wordle_levels_cebuano.json", "w", encoding="utf-8") as f:
    json.dump(make_level_file("Cebuano", topics), f, indent=1, ensure_ascii=False)
```

Part 5. Prefabs

Step 17. Create the 11 prefabs

Prefabs are saved in `Assets/Main/Prefabs`. Build the object in a scene, then drag it into the folder to make the prefab. The crossword has two versions: the plain one (older design) and **V2** (current design).

Prefab	Hierarchy	Scripts and notes
1 Menu/BackToMainMenuButton	BackToMainMenuButton (Image + Button) > Text (TMP)	The back button of the menu scenes. In each scene its OnClick is wired to `SceneNavigator.LoadMainMenu` on that scene's SceneManager object.
2 Wordle/WordleTile	WordleTile > Border (Image, rounded ring) > Fill (Image) > Letter (TMP)	**WordleTileView**: tile look, flip and shake animation. The verifier sets the Fill colour.
2 Wordle/WordleCell	WordleCell (Image) > Text	Plain cell of the first Wordle design (no script).
2 Wordle/Timer	Timer > Clock, background image timer - Image, Time (TMP)	**WordleTimer**: countdown; duration from WordleTimerSettings or code.
2 Wordle/DialogPanel	DialogPanel (Image) > Text (TMP)	**WordDefinitionDisplay** + **WordleDialogPanelLayoutApplier**: shows the clue.
3 Crossword/CrosswordCell	CrosswordCell > Fill (Image), Number (TMP), Letter (TMP)	**CrosswordCell**: letter, clue number, selected and wrong states.
3 Crossword/CrosswordCellV2	same as above	Current design of the cell (same script).
3 Crossword/BlackCell	BlackCell (Image)	Blocked cell (no script).
3 Crossword/CrosswordBlackCellV2	CrosswordBlackCellV2 (Image)	Current design of the blocked cell.
3 Crossword/CrosswordClueRow	CrosswordClueRow > Number (TMP), Clue (TMP)	**CrosswordClueRow**: one clue in the list.
3 Crossword/CrosswordClueRowV2	CrosswordClueRowV2 > Number, Clue, Done, check_a, check_b	Current design: strike-through and a check mark when solved.

The scenes created by the builders place most of their objects directly in the scene (not as prefabs), because each builder re-creates them. The Wordle and Crossword grids instantiate **WordleTile** and **CrosswordCellV2** at runtime through `WordleRowsColumnGenerator` and `CrosswordGridGenerator`.

Part 6. Scenes and games

Step 18. Scene basics (every scene)

1. File > New Scene (Basic 2D). Save it in the right `Scenes` folder with the exact name.
2. **Main Camera:** Orthographic, size 5, Clear Flags Solid Color, dark blue background.
3. **Canvas:** Render Mode Screen Space - Camera (drag the Main Camera in), Canvas Scaler Scale With Screen Size, 1920 x 1080, Match Expand. Add a **GraphicRaycaster**.
4. **EventSystem** with the Standalone Input Module (created with the first Canvas).
5. A **sky_base** Image that fills the canvas as the background colour, and the **Theme** object that tints it (the menu theme blends by time of day).
6. **SceneManager** empty object with `SceneNavigator` on menu scenes (buttons call it from OnClick).
7. Add each scene to **File > Build Settings** (step 29). `BackgroundMusic`, `SoundEffects` and `ScreenFit` need no scene objects.

Step 19. Main menu: `1 Main_menu`

Hierarchy (top levels)

```
EventSystem
SceneManager [SceneNavigator]
Main Camera
main_menu
  MenuMotion [MainMenuMotion]
  sky_base
    Text (TMP)
  MenuTheme [MainMenuTheme]
  MenuUI
    salita_pill (+1 children)
    boat_group (+1 children)
    library_card (+3 children)
    clouds_group (+3 children)
    wave_front2_crest
    play_card (+3 children)
    night_group (+42 children)
    tagline_pill (+1 children)
    wave_back_fill
    title_tile_K3 (+1 children)
    wave_front1_fill
    profile_chip (+3 children)
    gulls_group (+3 children)
    phrasebook_card [PhraseMenuButton] (+3 children)
    sky_top
    floating_tiles (+10 children)
    settings_chip (+1 children)
    sun_group (+2 children)
    title_tile_L1 (+1 children)
    ripples_group (+4 children)
    wave_back_crest
    title_tile_B4 (+1 children)
    settings_panel (+1 children)
    wave_front2_fill
    SettingsPanelController [SettingsPanel]
    title_tile_Y6 (+1 children)
    wave_front1_crest
    map
    wave_marks_group (+5 children)
    title_tile_A2 (+1 children)
    title_tile_A5 (+1 children)
    streak_chip [StreakChip] (+2 children)
```

- **MenuTheme** (`MainMenuTheme`) chooses the sky and sea colours from the clock: Morning 05:00-10:59, Noon 11:00-15:59, Afternoon 16:00-18:59, Night 19:00-04:59.
- **MenuMotion** (`MainMenuMotion`) animates tiles, title flip, waves, boat and clouds while playing.
- **MenuUI** holds the title tiles L A K B A Y, the three cards (Play, Library, Phrasebook), the day-streak chip (`StreakChip`), the Profile chip and the gear chip.
- Buttons: Play > `SceneNavigator.LoadLanguageSelectionMenu`; Library > `LoadLibraryMenu`; Phrasebook card has `PhraseMenuButton`; Profile > `LoadProfileIcon`.
- Run **Tools > Settings > Build gear button and settings window** to create the gear and the settings window (SettingsPanel below). Run **Tools > Phrasebook > Build Phrasebook scenes and menu card** to add the Phrasebook card.

The settings window: Sound effects On/Off and a Music volume slider (0 = music off), both saved.

Assets/Main/Scripts/1 Menu/SettingsPanel.cs

```
using TMPro;
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// SettingsPanel
/// The settings window of the main menu. The gear chip in the top bar (where the music chip was) opens
    it; it has two settings, both saved:
///  - Sound effects: an On / Off switch (SoundEffects.SetMuted), the button sounds of the whole game.
///  - Music volume: a slider from 0 to 100 % (BackgroundMusic.SetVolume). Dragging it to 0 turns the
    music off, anything above turns it
///    back on, so there is no separate music switch.
/// "Done", Esc and the gear chip close it. The window is built into the main menu scene by
    SettingsPanelBuilder (Tools > Settings); the
/// references are assigned there.
/// </summary>
public class SettingsPanel : MonoBehaviour
{
    [SerializeField] private GameObject panel;
    [SerializeField] private Button openButton;
    [SerializeField] private Button closeButton;

    [Header("Sound effects switch")]
    [SerializeField] private Button sfxButton;
    [SerializeField] private Image sfxTrack;
    [SerializeField] private TMP_Text sfxLabel;

    [Header("Music volume")]
    [SerializeField] private Slider volumeSlider;
    [SerializeField] private TMP_Text volumeLabel;

    [Header("Colours")]
    [SerializeField] private Color onColor = new Color32(93, 160, 42, 255);
    [SerializeField] private Color offColor = new Color32(185, 183, 172, 255);

    private void Awake()
    {
        if (openButton != null) openButton.onClick.AddListener(Toggle);
        if (closeButton != null) closeButton.onClick.AddListener(Close);
        if (sfxButton != null) sfxButton.onClick.AddListener(ToggleSfx);
        if (volumeSlider != null) volumeSlider.onValueChanged.AddListener(OnVolume);
        if (panel != null && Application.isPlaying) panel.SetActive(false);
    }

    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.Escape) && panel != null && panel.activeSelf)
        {
            Close();
        }
    }

    private void Toggle()
    {
        if (panel != null && panel.activeSelf) Close(); else Open();
    }

    /// <summary>Opens the window and shows the saved settings.</summary>
    public void Open()
    {
        Refresh();
        if (panel != null) panel.SetActive(true);
    }

    /// <summary>Closes the window.</summary>
    public void Close()
    {
```

```
        if (panel != null) panel.SetActive(false);
    }

    /// <summary>Reads the saved settings into the switch and the slider.</summary>
    public void Refresh()
    {
        bool sfxOn = !SoundEffects.Muted;
        if (sfxTrack != null) sfxTrack.color = sfxOn ? onColor : offColor;
        if (sfxLabel != null) sfxLabel.text = sfxOn ? "On" : "Off";
        float volume = BackgroundMusic.Muted ? 0f : BackgroundMusic.Volume;
        if (volumeSlider != null) volumeSlider.SetValueWithoutNotify(volume);
        ShowPercent(volume);
    }

    private void ToggleSfx()
    {
        bool nowMuted = !SoundEffects.Muted;
        SoundEffects.SetMuted(nowMuted);
        if (!nowMuted)
        {
            SoundEffects.Play("toggle_on");   // the press itself was silent while the effects were off
        }
        Refresh();
    }

    private void OnVolume(float value)
    {
        if (value <= 0.001f)
        {
            if (!BackgroundMusic.Muted) BackgroundMusic.SetMuted(true);
        }
        else
        {
            if (BackgroundMusic.Muted) BackgroundMusic.SetMuted(false);
            BackgroundMusic.SetVolume(value);
        }
        ShowPercent(value);
    }

    private void ShowPercent(float value)
    {
        if (volumeLabel != null) volumeLabel.text = Mathf.RoundToInt(value * 100f) + "%";
    }
}
```



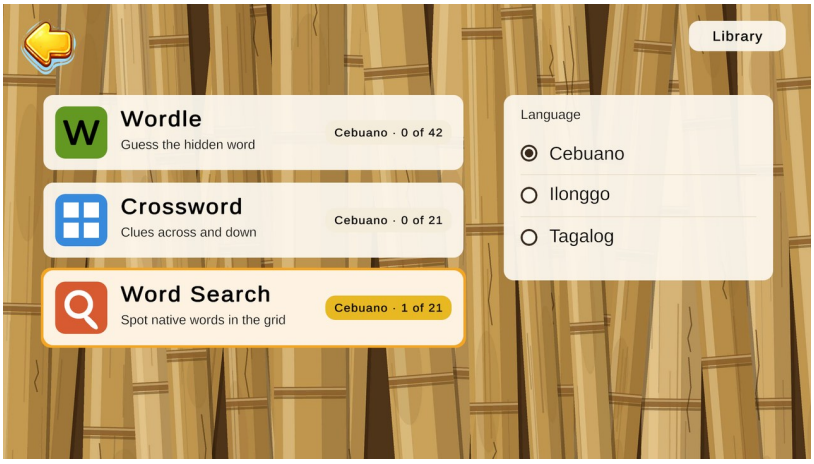
Main menu

Step 20. Language hub: `3 Language_Selection_Menu`

Hierarchy (top levels)

```
EventSystem
Main Camera
SceneManager [LanguageSelector, SceneNavigator]
Language_menu
  back_to_main_menu_button
    Text (TMP)
  HubUI
    hub_toast [HubToast] (+1 children)
    word_search_card_outline
    word_search_card (+4 children)
    crossword_card (+4 children)
    library_button (+1 children)
    wordle_card (+4 children)
    language_card (+6 children)
  background
    Text (TMP)
```

- **HubUI**: three game cards (Wordle, Crossword, Word Search) and a language panel with three Toggles in one **ToggleGroup**.
- On **SceneManager** add `LanguageSelector` (assign the three Toggles) and `SceneNavigator` (assign the LanguageSelector and the HubToast).
- Card buttons call `LoadRandomWordlePuzzle`, `LoadRandomCrosswordPuzzle` and `LoadRandomWordSearchPuzzle`. These store the language in the game's Session class and open that game's Start scene.
- Each card has a progress tag (`WordleHubTag`, `CrosswordHubTag`, `WordSearchHubTag`) showing "Cebuano · 4 of 42". Run the category builders (steps 21 to 23) to create and wire them.



Language hub

Step 21. Wordle: `Wordle - Start` and `Wordle - Play`

Scripts to add (all in `Scripts/2 Wordle`, full source in the project): `WordleRowsColumnGenerator` (builds rows x columns of tiles from the WordleTile prefab), `WordleKeyboardTyper` (typing into the grid with the physical keyboard or keys), `WordleWordVerifier` (compares a guess with the answer and colours tiles), `WordleTimer` and its settings, `WordleTileView`, `WordleWordValidator` (dictionary check), `WordleKeyButton`, `WordleGameController` (the screen), `WordleStartController`, `WordleCategoryBootstrap`, `WordleHubTag`.

How a guess is checked: the verifier makes two passes, first marking exact matches green, then marking letters that are in the word but in another place yellow (each answer letter used once), the rest gray. The row is revealed tile by tile with a flip.

Runs in Awake before everything else (execution order -200): reads the session, sets the word, the column count (4 to 7) and the 5:00 timer, and fills the category chip.

Assets/Main/Scripts/2 Wordle/WordleCategoryBootstrap.cs

```
using TMPro;
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// WordleCategoryBootstrap
/// Sets up the Wordle category play scene ("Wordle - Play") before the game starts. That one scene
    plays every category level: it reads
/// the chosen language, category and level from WordleSession, gives the word, its meaning and its
    clue to the WordleWordVerifier, sets
/// the number of columns to the word's length (4-7 letters), sets the 5:00 timer, switches the
    WordleGameController into category mode
/// and fills the category chip in the top bar (name, colour dot and "Level N of 6"). It runs very
    early (Awake, execution order -200),
/// before the verifier, the generator, the timer and the controller start. If the scene is opened
    directly without a session it plays
/// level 1 of the first category. References are assigned in the scene by WordleCategoryBuilder.
/// </summary>
[DefaultExecutionOrder(-200)]
public class WordleCategoryBootstrap : MonoBehaviour
{
    [SerializeField] private WordleWordVerifier verifier;
    [SerializeField] private WordleRowsColumnGenerator generator;
    [SerializeField] private WordleTimer timer;
    [SerializeField] private WordleGameController controller;
    [SerializeField] private int timerSeconds = 300;

    [Header("Category chip (top bar)")]
    [SerializeField] private Image categoryDot;
    [SerializeField] private TMP_Text categoryNameLabel;
    [SerializeField] private TMP_Text categoryLevelLabel;

    private void Awake()
    {
        int language = Mathf.Clamp(WordleSession.Language, 0, 2);
        int category = WordleSession.Category;
        int level = WordleSession.Level;

        WordleLevelDef def = WordleData.Level(language, category, level);
        if (def == null)
        {
            category = 0;
            level = 0;
            WordleSession.Category = 0;
            WordleSession.Level = 0;
            def = WordleData.Level(language, 0, 0);
        }
        if (def == null)
        {
            Debug.LogError("WordleCategoryBootstrap: no Wordle level data (Data/Wordle/Levels, see
                Resources/LevelDataRegistry.asset).");
            return;
        }

        if (verifier != null) verifier.SetTarget(def.word, def.meaning, def.clue);
        if (generator != null) generator.columns = Mathf.Max(1, def.word.Length);
        if (timer != null) timer.OverrideDuration(timerSeconds);
        if (controller != null) controller.ConfigureCategory(language, category, level);

        WordleCategoryDef categoryDef = WordleData.Category(language, category);
        if (categoryDot != null) categoryDot.color = WordleData.CategoryColor(category);
        if (categoryNameLabel != null && categoryDef != null) categoryNameLabel.text = categoryDef.name;
        if (categoryLevelLabel != null && categoryDef != null)
        {
            categoryLevelLabel.text = "Level " + (level + 1) + " of " + categoryDef.levels.Count;
        }
    }
```

```
}
```

One key of the on-screen keyboard.

Assets/Main/Scripts/2 Wordle/WordleKeyButton.cs

```
using TMPro;
using UnityEngine;
using UnityEngine.UI;

/// <summary>What a key on the on-screen Wordle keyboard does.</summary>
public enum WordleKeyKind
{
    Letter,
    Enter,
    Backspace
}

/// <summary>
/// WordleKeyButton
/// One key of the on-screen Wordle keyboard (put on each key object together with a Button and an
    Image).
/// WordleGameController listens to Pressed and types the letter, submits the row or deletes a letter.
    Keys stay plain
/// after a guess (so the keypad does not give away right or wrong letters); if
    WordleGameController.colorKeyboardKeys is
/// turned on, the controller colours letter keys green / yellow / gray through SetState instead.
    Letter keys remember
/// the best state they ever had (green beats yellow beats gray).
/// </summary>
public class WordleKeyButton : MonoBehaviour
{
    public WordleKeyKind kind;
    public char letter;
    public Image background;
    public TMP_Text label;

    [SerializeField] private Color idleColor = new Color32(253, 248, 238, 255);
    [SerializeField] private Color idleText = new Color32(59, 42, 26, 255);

    /// <summary>Raised when the player taps the key.</summary>
    public event System.Action<WordleKeyButton> Pressed;

    private int stateRank;

    private void Awake()
    {
        Button button = GetComponent<Button>();
        if (button != null)
        {
            button.onClick.AddListener(OnClick);
        }
    }

    private void OnClick()
    {
        if (Pressed != null)
        {
            Pressed(this);
        }
    }

    /// <summary>Colours the key. rank: 1 = not in the word, 2 = wrong position, 3 = correct. Lower
        ranks never overwrite higher ones.</summary>
    public void SetState(int rank, Color color)
    {
        if (kind != WordleKeyKind.Letter || rank <= stateRank)
        {
            return;
        }
        stateRank = rank;
        if (background != null) background.color = color;
        if (label != null) label.color = Color.white;
```

```
    }

    /// <summary>Returns the key to its plain look.</summary>
    public void ResetState()
    {
        stateRank = 0;
        if (background != null) background.color = idleColor;
        if (label != null) label.color = idleText;
    }
}
```

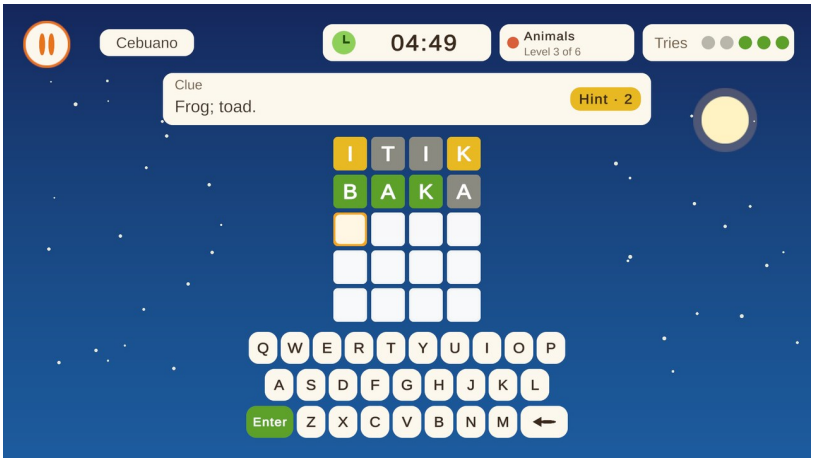
Build the scenes: run **Tools > Wordle > Build category scenes and hub tag**. It creates `Wordle - Start` (back button, title chip, language chip, Random word button, seven topic cards with solved tag, level stepper and Play button) and `Wordle - Play` (one scene for every level), saves them in `Scenes/2 Wordle`, adds them to Build Settings and wires the hub tag.

`Wordle - Play` hierarchy:

Hierarchy (top levels)

```
Main Camera
SceneManager [SceneNavigator]
Canvas
  WordleOverlays
    pause_panel (+1 children)
    result_panel (+2 children)
  Timer [WordleTimer]
  sky_base
  WordleController [WordleGameController, WordleCategoryBootstrap]
  WordleGenerator [WordleRowsColumnGenerator, WordleKeyboardTyper, WordleWordVerifier]
  WordleUI
    night_group (+42 children)
    sun_group (+2 children)
    Board (+3 children)
    category_chip (+3 children)
    toast [HubToast] (+1 children)
    sky_top
    pause_button (+3 children)
    clouds_group (+4 children)
    language_chip (+1 children)
    timer_pill (+2 children)
    tries_pill (+6 children)
  WordleTheme [MainMenuTheme]
EventSystem
```

Setting on WordleGameController	Value
Hints per puzzle	2
Rows (tries)	5 (on WordleRowsColumnGenerator); columns are set from the word length
Timer	300 seconds (WordleCategoryBootstrap)
Color Keyboard Keys	off
Reject Unknown Words	off (dictionary file lacks some valid words)



Wordle play screen

Step 22. Crossword: `Crossword - Start` and `Crossword - Play`

Scripts (in `Scripts/3 Crossword and Gameplay`): `CrosswordPuzzleData` and `CrosswordCategoryData` (printed in step 14), `CrosswordGridGenerator` (builds the cells from a puzzle: a `CrosswordCell` where a word's letter sits, a black cell elsewhere), `CrosswordCell`, `CrosswordClueRow`, `CrosswordCluePanel` (Across and Down lists, active-clue banner), `CrosswordInputController` (active word and cell, typing, arrow keys, hint, check, reveal), `CrosswordGameController` (the screen: clue card, tabs, stopwatch, keypad switch, pause, result), `CrosswordCategoryBootstrap`, `CrosswordStartController`, `CrosswordHubTag`, `CrosswordTextStyle` (one place for all text sizes).

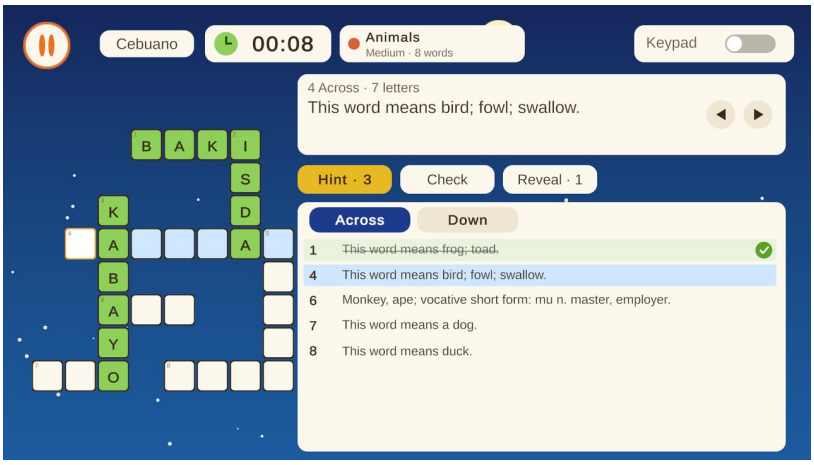
Build the scenes: run **Tools > Crossword > Archive current crossword and build category scenes** (creates `Crossword - Start` and `Crossword - Play`, adds them to Build Settings). `Crossword - Play` also needs a **GameController** object with `CrosswordInputController` and `CrosswordCluePanel`.

Hierarchy (top levels)

```
Canvas
  CrosswordTheme [MainMenuTheme]
  CrosswordUI
    toast [HubToast] (+1 children)
    timer_pill (+2 children)
    night_group (+41 children)
    sun_group (+2 children)
    sky_top
    pause_button (+3 children)
    category_chip (+3 children)
    language_chip (+1 children)
    keypad_switch (+2 children)
    clouds_group (+2 children)
    Board (+7 children)
  CrosswordController [CrosswordGameController, CrosswordCategoryBootstrap]
  sky_base
  CrosswordOverlays
    result_panel (+2 children)
    pause_panel (+1 children)
SceneManager [SceneNavigator]
Main Camera
EventSystem [CrosswordAnswerKey]
GameController [CrosswordInputController, CrosswordCluePanel]
```

Rule	Value
Sizes	Small 5 words, Medium 8, Large 11

Rule	Value
Hints	3 per puzzle (reveal a letter); Reveal once (fills the active word); Check marks wrong letters
Limits	No tries, no time limit; a stopwatch
Order	Small, Medium, Large of a topic, then the next topic



Crossword play screen

Step 23. Word Search: `WordSearch - Start` and `WordSearch - Play`

Scripts (`Scripts/4 Word Search`): `WordSearchData`, `WordSearchGenerator` (printed in step 15), `WordSearchSession` (language, category, puzzle and mode, the scene names and the solved store), `WordSearchBoardView` (the 10 x 10 labels and the drag selection that snaps to the nearest straight line), `WordSearchGameController`, `WordSearchStartController`, `WordSearchHubTag`.

Run **Tools > Word Search > Build Start and Play scenes**. The play scene has the board (100 letter labels), the word list (shortest first), the top bar (pause, language chip, stopwatch, "Found N of M"), a Hint button (3 per puzzle: a ring pulses on the first letter of an unfound word), pause window and result card.

Hierarchy (top levels)

```
Main Camera
Canvas
  WordSearchTheme [MainMenuTheme]
  WordSearchController [WordSearchGameController]
  WordSearchOverlays
    result_panel (+2 children)
    pause_panel (+1 children)
  sky_base
  WordSearchUI
    pause_button (+3 children)
    language_chip (+1 children)
    night_group (+41 children)
    sun_group (+2 children)
    clouds_group (+2 children)
    Board (+3 children)
    timer_pill (+2 children)
    found_pill (+1 children)
    sky_top
    toast [HubToast] (+1 children)
EventSystem
```



Word Search play screen

Step 24. Library: `4 Library_menu`

The Library is **built in code at runtime**: `LibraryController` creates the whole screen on top of a background and a back button, so the scene is tiny.

Hierarchy (top levels)

```
EventSystem
SceneManager [SceneNavigator]
LibraryController [LibraryController]
Main Camera
library_menu
  back_to_main_menu_button
    Text (TMP)
  background
    Text (TMP)
  LibraryUI
    SearchBox (+3 children)
    SideCard (+4 children)
    Pill (+1 children)
    Results (+1 children)
    Pill (+1 children)
    Compare
    Notebook [LibraryNotebook] (+25 children)
    TotalPill (+1 children)
    Hint (+2 children)
    Pill (+1 children)
    TitlePill (+1 children)
    Pill (+1 children)
    Pill (+1 children)
    DetailCard (+11 children)
```

1. Create the scene with a bamboo background Image and the back button prefab.
2. Add an empty **LibraryController** object with `LibraryController.cs` and assign the three dictionary JSON files (cebuano, hiligaynon, tagalog) to its TextAsset fields.
3. Add the supporting scripts: `LibraryData` (parsing, cleaning, ranked search, did-you-mean, word of the day, favorites and recent searches in PlayerPrefs), `LibraryNotebook` (virtualised A to Z list), `LibraryUiKit` (colours, procedural sprites, factory methods), `LibraryLetterStrip` and `LibraryScrollTrack` (pointer handlers), `LibraryFavoritesStore` (below).

Assets/Main/Scripts/5 Library/LibraryFavoritesStore.cs

```
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// LibraryFavoritesStore
/// Small static helper that reads and writes the Library's favorite words directly in PlayerPrefs,
    using the exact same
/// key and format as LibraryData ("library.favorites.v1", entries "LanguageIndex:WORD" joined with
    "|"; language index 0 =
/// Cebuano, 1 = Hiligaynon/Ilonggo, 2 = Tagalog). It lets other screens (the Crossword result card)
    star a word without
/// loading the whole dictionary; the Library shows the favorite the next time it opens.
/// </summary>
public static class LibraryFavoritesStore
{
    private const string FavoritesKey = "library.favorites.v1";

    private static string KeyFor(int language, string word)
    {
        return language + ":" + (word ?? string.Empty).Trim().ToUpperInvariant();
    }

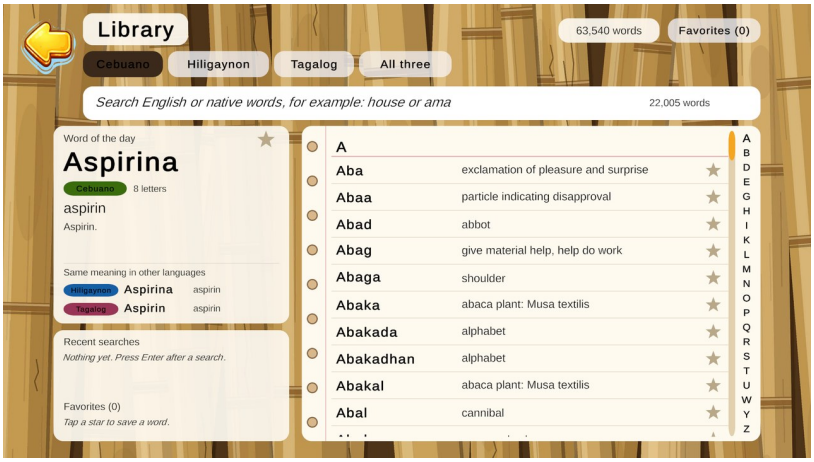
    private static List<string> Load()
    {
        var list = new List<string>();
        string saved = PlayerPrefs.GetString(FavoritesKey, string.Empty);
        if (saved.Length > 0)
        {
            foreach (string part in saved.Split('|'))
            {
                if (part.Length > 0)
                {
                    list.Add(part);
                }
            }
        }
        return list;
    }

    /// <summary>How many words are saved as favorites (all languages).</summary>
    public static int Count()
    {
        return Load().Count;
    }

    /// <summary>True if this word is saved as a favorite in that language.</summary>
    public static bool IsFavorite(int language, string word)
    {
        return Load().Contains(KeyFor(language, word));
    }

    /// <summary>Adds the word to the favorites, or removes it if it was already there. Returns the new
        state (true = favorite).</summary>
    public static bool Toggle(int language, string word)
    {
        List<string> list = Load();
        string key = KeyFor(language, word);
        bool nowFavorite;
        if (list.Remove(key))
        {
            nowFavorite = false;
        }
        else
        {
            list.Add(key);
            nowFavorite = true;
        }
```

```
        PlayerPrefs.SetString(FavoritesKey, string.Join("|", list.ToArray()));
        PlayerPrefs.Save();
        return nowFavorite;
    }
}
```



Library

Step 25. Learn cards: `Learn - Cards`

Run **Tools > Learn > Build Learn scene and buttons**. It creates the scene and adds a **Learn** button (`LearnTopicButton`) to each topic card of the three start screens. `LearnController` shows one card per word using `WordSearchData` (the three sets A, B, C of a topic), with Back, Next and Start puzzle.

Assets/Main/Scripts/6 Learn/LearnSession.cs

```
using UnityEngine;
using UnityEngine.SceneManagement;

/// <summary>Which game's topic card opened the Learn screen (and where "Back" and "Start puzzle"
    lead).</summary>
public enum LearnGame
{
    Wordle,
    Crossword,
    WordSearch
}

/// <summary>
/// LearnSession
/// Static hand-over for the Learn screen (scene "Learn - Cards"): which game opened it, the language,
    the topic (category index) and the set
/// (0, 1, 2 = A, B, C, the same word sets as the three Word Search puzzles of the topic). The Learn
    button on a topic card (LearnTopicButton)
/// fills it through Open(), LearnController reads it. Language index everywhere: 0 Cebuano, 1 Ilonggo,
    2 Tagalog.
/// </summary>
public static class LearnSession
{
    public const string LearnScene = "Learn - Cards";

    public static LearnGame Game;
    public static int Language;
    public static int Category;
    public static int Set;

    /// <summary>Stores what to learn and opens the Learn screen.</summary>
    public static void Open(LearnGame game, int language, int category)
    {
        Game = game;
        Language = Mathf.Clamp(language, 0, 2);
        Category = Mathf.Max(0, category);
        Set = 0;
        SceneManager.LoadScene(LearnScene);
    }

    /// <summary>The start (topic) screen of the game that opened the Learn screen.</summary>
    public static string ReturnScene()
    {
        switch (Game)
        {
            case LearnGame.Crossword: return CrosswordSession.StartScene;
            case LearnGame.WordSearch: return WordSearchSession.StartScene;
            default: return WordleSession.StartScene;
        }
    }
}
```

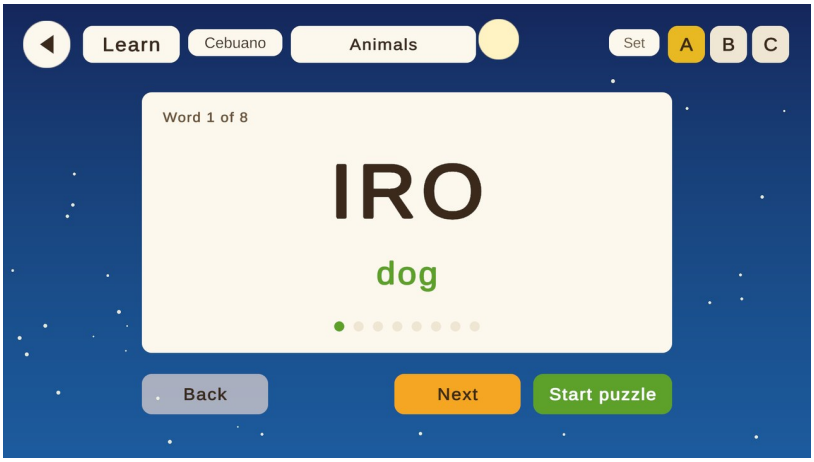
Assets/Main/Scripts/6 Learn/LearnTopicButton.cs

```
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// LearnTopicButton
/// Put on the "Learn" button of a topic card on a game's start screen (Wordle - Start, Crossword -
    Start, WordSearch - Start). Pressing it opens
/// the Learn screen for that topic: it takes the language the game's session holds, and the topic
    (category index) set on this component.
/// Needs a Button on the same object. The Learn buttons are added to the start scenes by
    LearnSceneBuilder (Tools > Learn).
/// </summary>
[RequireComponent(typeof(Button))]
public class LearnTopicButton : MonoBehaviour
{
    public LearnGame game;
    public int category;

    private void Awake()
    {
        GetComponent<Button>().onClick.AddListener(Open);
    }

    /// <summary>Opens the Learn screen for this topic.</summary>
    public void Open()
    {
        int language;
        switch (game)
        {
            case LearnGame.Crossword: language = CrosswordSession.Language; break;
            case LearnGame.WordSearch: language = WordSearchSession.Language; break;
            default: language = WordleSession.Language; break;
        }
        LearnSession.Open(game, language, category);
    }
}
```



Learn card

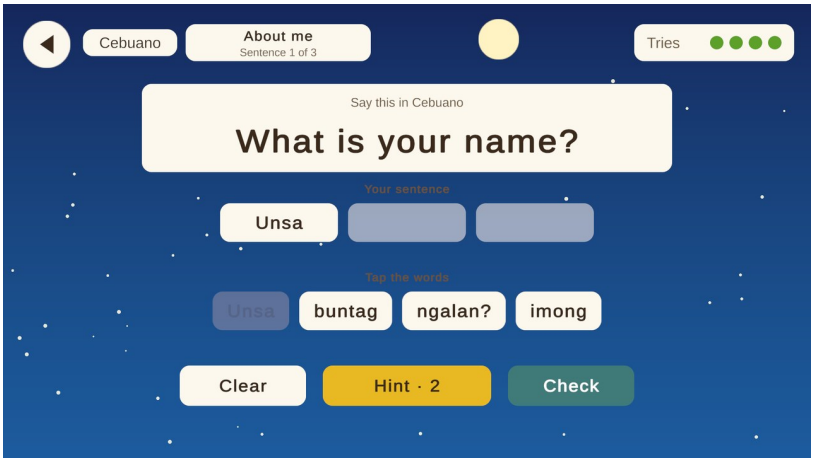
Step 26. Phrasebook: Topics, Cards, Builder

Run **Tools > Phrasebook > Build Phrasebook scenes and menu card**. It points the registry's Phrasebook Files at the JSON, creates the three scenes (`Phrasebook - Topics`, `Phrasebook - Cards`, `Phrasebook - Builder`) and the Phrasebook card on the main menu. Scripts: `PhraseData` and `PhraseMenuButton` (step 16), `PhraseSession`, `PhraseTopicsController` (language chips and seven topic cards), `PhraseCardsController` (one card per phrase), `PhraseBuilderController`.

Sentence builder rules: the tile bank holds the phrase's words plus one wrong word from another phrase, shuffled. Tap a tile to fill the next slot; tap a filled slot to take the word back; Clear empties. Check: right = the phrase is marked learned; wrong = one of 4 tries lost. Hint puts the next correct word in place (2 per sentence). Tiles and slots are copies of hidden `tile_template` and `slot_template` objects.

Hierarchy (top levels)

```
EventSystem
Main Camera
Canvas
  SentenceBuilderUI
    check_button (+1 children)
    Text
    Text
    result_panel (+1 children)
    sun_group (+2 children)
    bank_row (+5 children)
    Text
    answer_row (+4 children)
    language_chip (+1 children)
    hint_button (+1 children)
    topic_chip (+2 children)
    clear_button (+1 children)
    clouds_group (+2 children)
    night_group (+41 children)
    back_button (+1 children)
    sky_top
    tries_chip (+5 children)
    prompt_card (+2 children)
  PhrasebookTheme [MainMenuTheme]
  sky_base
  PhraseBuilderController [PhraseBuilderController]
```



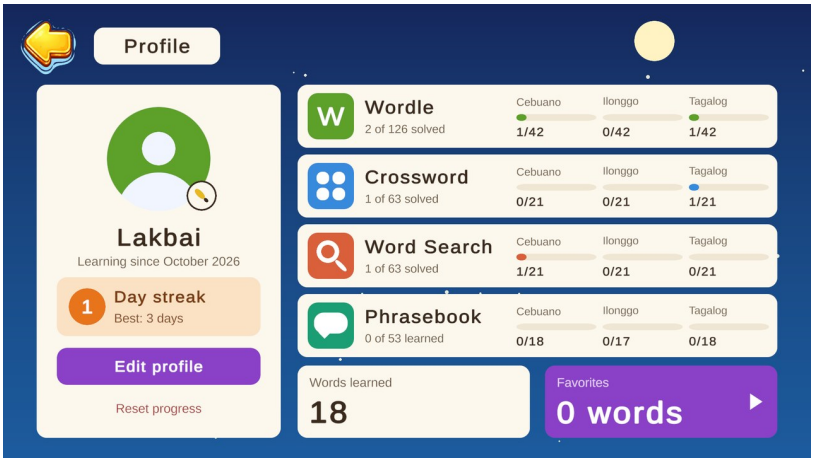
Sentence builder

Step 27. Profile and settings

Run **Tools > Profile > Rebuild design in open scene** with `2 Profile Icon` open. `ProfileScreenController` reads `PlayerDatabase`, `StreakTracker` and the four progress stores and shows: avatar (10 colours) and name, "Learning since", day streak and best streak, a row with a bar per language for Wordle, Crossword, Word Search and Phrasebook, Words learned, Favorites, the Edit profile dialog and the Reset progress dialog.

Hierarchy (top levels)

```
PlayerDatabase [PlayerDatabase]
Main Camera
Canvas
  sky_base
  back_to_main_menu_button
    Text (TMP)
  ProfileUI
    crossword_card (+15 children)
    clouds_group (+2 children)
    sun_group (+2 children)
    favorites_card (+3 children)
    night_group (+41 children)
    wordsearch_card (+15 children)
    wordle_card (+15 children)
    words_learned_card (+2 children)
    title_pill (+1 children)
    phrasebook_card (+15 children)
    profile_card (+7 children)
    sky_top
  ProfileTheme [MainMenuTheme]
  ProfileController [ProfileScreenController]
  ProfileOverlays
    reset_panel (+1 children)
    edit_panel (+1 children)
SceneManager [SceneNavigator]
EventSystem
```



Profile screen

Part 7. Audio code

Step 28. Music, button sounds and Tap to hear

BackgroundMusic (`Scripts/Audio/BackgroundMusic.cs`, 285 lines) creates itself, picks a track by scene name, fades the old track out in 0.08 s and the new one in over 0.3 s, keeps playing when moving between scenes with the same track, and saves volume and mute. Track by scene: `1 Main_menu` Lakbay, `2 Profile Icon` Ako Ni, `3 Language_Selection_Menu` Tulay, `4 Library_menu` Tahimik, any Wordle scene Hula-Hula, any Crossword scene Pahalang Pababa, any Word Search scene Pangita.

SoundEffects plays a sound for every button without setup: on each press it asks the EventSystem what is under the pointer and plays the clip chosen by the button's name.

Assets/Main/Scripts/Audio/SoundEffects.cs

```
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.UI;

/// <summary>
/// SoundEffects
/// Plays a sound for every button in the game. It creates itself before the first scene loads
    (RuntimeInitializeOnLoadMethod), survives scene
/// loads (DontDestroyOnLoad) and needs nothing placed in any scene, like BackgroundMusic. Every time
    the player presses on the screen it asks the
/// EventSystem what is under the pointer; if that is a Button (or a Toggle) that can be used, it plays
    a sound from
/// Assets/Main/Resources/Audio SFX, chosen by the button:
///  - Back arrows, "Back to ..." and "topics" buttons: ui_back; Cancel: ui_back
///  - the pause button and the settings gear: ui_open; Resume, Done and close buttons: ui_close
///  - Hint buttons: hint
///  - Wordle keypad: key_tap_1/2/3 (random) for letters, key_delete for Backspace, key_enter for Enter
///  - the Sound effects switch in the settings window plays toggle_on itself when it turns the effects
    back on
///  - every other button, including buttons the games create while they run (tiles, list rows, Library
    items): ui_click
/// Because it looks at the screen and not at each button, new buttons get their sound without any
    setup. SoundEffects.Play("clip name") plays
/// any clip of that folder from code. Sound effects can be switched off with
    SoundEffects.SetMuted(true) (saved in PlayerPrefs "SfxMuted");
/// there is no switch for it in the menus yet. The sounds are described in Resources/Audio SFX/README
    - Sound Effects.md.
/// </summary>
public class SoundEffects : MonoBehaviour
{
    private const string Folder = "Audio SFX/";
    private const string MutedKey = "SfxMuted";
    private const float Volume = 0.9f;

    private static SoundEffects _instance;
    private readonly Dictionary<string, AudioClip> _clips = new Dictionary<string, AudioClip>();
    private AudioSource _source;
    private bool _muted;

    /// <summary>The name of the clip played last (for testing), for example "ui_click".</summary>
    public static string LastPlayed { get; private set; }

    /// <summary>True when the player turned the sound effects off.</summary>
    public static bool Muted
    {
        get { return _instance != null ? _instance._muted : PlayerPrefs.GetInt(MutedKey, 0) == 1; }
    }

    /// <summary>Turns the sound effects off (true) or on (false) and saves the choice.</summary>
    public static void SetMuted(bool muted)
    {
        PlayerPrefs.SetInt(MutedKey, muted ? 1 : 0);
        if (_instance != null)
        {
            _instance._muted = muted;
        }
    }

    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]
    private static void Create()
    {
        if (_instance != null)
        {
            return;
        }
        var go = new GameObject("SoundEffects");
```

```
        DontDestroyOnLoad(go);
        _instance = go.AddComponent<SoundEffects>();
        _instance._source = go.AddComponent<AudioSource>();
        _instance._source.playOnAwake = false;
        _instance._source.spatialBlend = 0f;
        _instance._muted = PlayerPrefs.GetInt(MutedKey, 0) == 1;
    }

    /// <summary>Plays one clip of Resources/Audio SFX by name (for example "ui_click"), unless the
        effects are muted.</summary>
    public static void Play(string clipName, float volume = 1f)
    {
        if (_instance == null)
        {
            Create();
        }
        if (_instance._muted)
        {
            return;
        }
        AudioClip clip;
        if (!_instance._clips.TryGetValue(clipName, out clip))
        {
            clip = Resources.Load<AudioClip>(Folder + clipName);
            _instance._clips[clipName] = clip;
        }
        if (clip != null)
        {
            LastPlayed = clipName;
            _instance._source.PlayOneShot(clip, Volume * volume);
        }
    }

    private void Update()
    {
        if (Input.GetMouseButtonDown(0))
        {
            PlayAt(Input.mousePosition);
        }
    }

    /// <summary>Plays the sound of the button under a screen position (nothing if there is none). Used
        on every press.</summary>
    public static void PlayAt(Vector2 screenPosition)
    {
        EventSystem es = EventSystem.current;
        if (es == null)
        {
            return;
        }
        var data = new PointerEventData(es) { position = screenPosition };
        var hits = new List<RaycastResult>();
        es.RaycastAll(data, hits);
        if (hits.Count == 0)
        {
            return;
        }
        // only the topmost UI element counts: a panel in front of a button blocks it
        Selectable selectable = hits[0].gameObject.GetComponentInParent<Selectable>();
        if (selectable == null || !selectable.IsInteractable())
        {
            return;
        }
        string clip = ClipFor(selectable);
        if (clip != null)
        {
            Play(clip);
        }
    }
```

```
    /// <summary>The sound for a button, by what it is; null = silent (the button makes its own
        sound).</summary>
    private static string ClipFor(Selectable selectable)
    {
        var key = selectable.GetComponent<WordleKeyButton>();
        if (key != null)
        {
            if (key.kind == WordleKeyKind.Backspace) return "key_delete";
            if (key.kind == WordleKeyKind.Enter) return "key_enter";
            return "key_tap_" + Random.Range(1, 4);
        }
        string name = selectable.gameObject.name.ToLowerInvariant();
        if (name.Contains("back") || name.Contains("topics") || name.Contains("cancel")) return
            "ui_back";
        if (name.Contains("pause") || name.Contains("settings")) return "ui_open";
        if (name.Contains("resume") || name.Contains("close") || name.Contains("done")) return
            "ui_close";
        if (name.Contains("hint")) return "hint";
        return "ui_click";
    }
}
```

WordAudio and **WordAudioPlayer** (`Scripts/Audio`) implement Tap to hear: a button plays `Resources/Audio Words/<Language>/<name>.wav` and shows "No recording for this word yet" when the file is missing. `Tools > Audio > Add Tap to hear buttons` installs the speaker buttons; **Hide Tap to hear buttons** switches them off (the current state, because no recordings exist).

Part 8. Build and test

Step 29. Build Settings and the Windows build

1. File > Build Settings. Add **only the 15 playable scenes**, in this order: `1 Main_menu`, `2 Profile Icon`, `3 Language_Selection_Menu`, `4 Library_menu`, `Wordle - Start`, `Wordle - Play`, `Crossword - Start`, `Crossword - Play`, `WordSearch - Start`, `WordSearch - Play`, `Learn - Cards`, `Phrasebook - Topics`, `Phrasebook - Cards`, `Phrasebook - Builder` (the old `CrossWord - Generate` scene is optional).
2. Do not add archived scenes. (The current project's Build Settings still lists 60 archived scenes; remove them for a lean build.)
3. Platform: **Windows, Mac, Linux** > Target Platform Windows, Architecture x86_64.
4. Click Build and choose a folder. The result is `Lakbay-Salita.exe` with its `_Data` folder; keep them together.
5. Run the .exe. Check that the window cannot be dragged larger than 1024 x 768 or smaller than 800 x 600, and that maximize gives a 1920 x 1080 full screen window (this behaviour has been checked in the editor only).

Step 30. Test checklist

Area	Check
Menu	Title tiles flip; Play, Library, Phrasebook, Profile and the gear open the right screens; the sky matches the time of day
Settings	Sound effects off silences button sounds; music slider 0 turns the music off; both are remembered after restart
Hub	Language radio buttons change the progress tags; each game card opens its Start scene in that language
Wordle	Type with the keyboard and the keypad; a short word is rejected with a shake; a hint names a letter and column; win shows the result card; solved count rises
Crossword	Clicking a clue selects its word; Hint, Check and Reveal work; every word solved shows the result card
Word Search	Drag selects in a straight line; found words are struck through; Hint pulses a ring; 'Found N of M' updates
Learn	Cards show the word and its meaning; Start puzzle opens the right game
Phrasebook	Language chips switch; Build accepts the right order and marks the phrase learned; wrong costs a try
Library	Search 'house' and 'balay'; stars save favorites; A to Z strip jumps
Profile	Counts and bars match solved puzzles; Reset progress clears them
Window	800 x 600 and 1024 x 768 show no overlap (letterbox bars in 4:3); Esc pauses games
Data	Open the Console: no red errors, no 'LevelDataRegistry ... not assigned'

Appendix A. Every script

All 97 C# scripts of the project with their size and purpose (taken from each script's own summary comment). Paths are relative to `Assets/Main/Scripts/`.

Script	Lines	Purpose
`1 Menu/DeleteProfileButton.cs`	16	Wired to the Profile screen's delete_button. Placeholder for future use (e.g. resetting PlayerData / deleting the save file) — intentionally left unimplemented since...
`1 Menu/Editor/ProfileDesignBuilder.cs`	569	Editor tool that rebuilds the "2 Profile Icon" scene in the redesigned look, as real saved scene objects (so the design is visible in the editor before pressing...
`1 Menu/Editor/SettingsPanelBuilder.cs`	204	Editor tool that replaces the music chip of the main menu with a gear button and builds the settings window behind it. Menu: Tools > Settings > "Build gear button and...
`1 Menu/HubToast.cs`	41	A small pop-up message on the Language Selection hub (for example "Word Search is coming soon"). Put it on a GameObject that is inactive by default and assign its...
`1 Menu/LanguageSelector.cs`	31	Reads which language Toggle is checked on the hub (PuzzleLanguage enum).
`1 Menu/MainMenuMotion.cs`	165	Brings the Main Menu to life while the game is running (everything stays still in the editor so the scene is easy to edit): - Floating letter tiles bob, sway and rock...
`1 Menu/MainMenuTheme.cs`	369	Gives the Main Menu a different sky and sea for each part of the day, chosen from the device clock: Morning 05:00-10:59, Noon 11:00-15:59, Afternoon 16:00-18:59,...
`1 Menu/ProfileDisplay.cs`	39	Pushes the current player's per-game progress from PlayerDatabase onto the Profile screen's TMP text elements ("... data score - Text (TMP)" objects in the "2 Profile...
`1 Menu/ProfileNameEditor.cs`	72	Wired to the Profile screen's edit_button. On Start, shows the player's current name (from PlayerDatabase.PlayerNameOrId) in the "User Name" Text (TMP). Clicking...
`1 Menu/ProfileScreenController.cs`	310	Drives the redesigned Profile screen (scene "2 Profile Icon"). It reads everything from PlayerDatabase and StreakTracker and writes it onto the screen: avatar and...
`1 Menu/SettingsPanel.cs`	108	The settings window of the main menu. The gear chip in the top bar (where the music chip was) opens it; it has two settings, both saved: - Sound effects: an On / Off...
`1 Menu/StreakChip.cs`	27	Shows the player's current day-streak (from StreakTracker) in the Main Menu's top-left chip. Put it on the chip object and assign the number label (the orange...
`2 Wordle/Editor/WordleCategoryBuilder.cs`	513	Editor tool that sets up the Wordle categories as real saved scenes, in the same look as the menus and the redesigned Wordle: - "Wordle - Start": back button, Wordle...

Script	Lines	Purpose
2 Wordle/Editor/WordleDesignBuilder.cs	703	Editor tool that rebuilds a Wordle puzzle scene in the redesigned look, as real saved scene objects (so the design is visible in the editor before pressing Play). It...
2 Wordle/Editor/WordleRowsColumnGeneratorEditor.cs	27	Custom Inspector for WordleRowsColumnGenerator: shows the default fields (rows, columns, spacing, cell prefab, grid parent, cell size) plus a "Generate" button that...
2 Wordle/WordDefinitionDisplay.cs	50	Shows the current puzzle's word definition (from WordleWordVerifier) in the Dialog Panel's TMP text. Refreshes automatically in the Editor via OnValidate whenever the...
2 Wordle/WordleCategoryBootstrap.cs	62	Sets up the Wordle category play scene ("Wordle - Play") before the game starts. That one scene plays every category level: it reads the chosen language, category and...
2 Wordle/WordleData.cs	141	The data model and loader for the category levels of Wordle. The levels live in one JSON file per language,...
2 Wordle/WordleDialogPanelLayout.cs	41	Shared layout and text settings (padding, size, style, colour) for the Wordle Dialog Panel; one asset for every Wordle scene.
2 Wordle/WordleDialogPanelLayoutApplier.cs	98	Applies the shared dialog-panel layout (padding, font size, style, colour) to the Dialog Panel of a scene.
2 Wordle/WordleGameController.cs	898	Runs the redesigned Wordle screen on top of the existing WordleKeyboardTyper / WordleWordVerifier / WordleTimer: - shows the language, the clue (definition from the...
2 Wordle/WordleHubTag.cs	61	The small tag on the Wordle card of the Language Selection hub. It shows how many Wordle levels the player has solved in the language that is selected on the hub,...
2 Wordle/WordleKeyButton.cs	72	One key of the on-screen Wordle keyboard (put on each key object together with a Button and an Image). WordleGameController listens to Pressed and types the letter,...
2 Wordle/WordleKeyboardTyper.cs	295	Lets you type letters directly into the grid generated by WordleRowsColumnGenerator without clicking into a cell first — mirrors how...
2 Wordle/WordleRowsColumnGenerator.cs	125	Generates a rows x columns grid of letter tiles, configured entirely from the Inspector: rows, columns, and per-cell pixel spacing are plain serialized fields (edited...
2 Wordle/WordleSceneTimerDurations.cs	60	Central list of starting timer durations, one entry per Wordle scene (matched by scene name).
2 Wordle/WordleSession.cs	111	Static hand-over between the Wordle category scenes: which language, category and level (0-based) to play. The language hub sets the language, the start screen sets...
2 Wordle/WordleStartController.cs	206	Runs the Wordle start screen (scene "Wordle - Start"): the language chosen on the hub, a "Random word" button and one card per category (Animals, Food and drink,...
2 Wordle/WordleTextStyle.cs	38	Shared font and size settings for the Wordle grid cells and dialog text; one asset for all Wordle scenes.
2 Wordle/WordleTextStyleApplier.cs	85	Applies the shared WordleTextStyle to the grid cells and the dialog text of a scene.

Script	Lines	Purpose
2 Wordle/WordleTileView.cs	215	Look and small animations of one letter tile in the redesigned Wordle grid. A tile is a rounded border Image with a rounded fill Image inside it (the fill is the...
2 Wordle/WordleTimer.cs	248	Countdown timer of a Wordle puzzle: duration from a shared asset or from code, shows MM:SS, stops when the word is solved.
2 Wordle/WordleTimerSettings.cs	42	Shared timer settings (text style, start delay, position) for every Wordle scene.
2 Wordle/WordleWordValidator.cs	91	Answers "is this a real word?" for the Wordle screen, using the same dictionary JSON files as the Library (Assets/Main/Data/Language Dictionary/cebuano.json,...
2 Wordle/WordleWordVerifier.cs	275	Checks a completed row against the target word with two-pass scoring (exact matches green, letters elsewhere yellow, the rest gray) and colours the tiles.
3 Crossword/CrosswordCategoryBootstrap.cs	57	Sets up the Crossword category play scene ("Crossword - Play") before the game starts. That one scene plays every category puzzle: it reads the chosen language,...
3 Crossword/CrosswordCategoryData.cs	151	The data model and loader for the category crosswords. The puzzles live in one JSON file per language,...
3 Crossword/CrosswordGameController.cs	843	Runs the redesigned Crossword screen on top of the existing CrosswordInputController / CrosswordCluePanel: - the active clue card (number, direction, letter count,...
3 Crossword/CrosswordHubTag.cs	60	The small tag on the Crossword card of the Language Selection hub. It shows how many Crossword puzzles the player has solved in the language that is selected on the...
3 Crossword/CrosswordSession.cs	111	Static hand-over between the Crossword category scenes: which language, category and size (0 Small, 1 Medium, 2 Large) to play. The language hub sets the language,...
3 Crossword/CrosswordStartController.cs	209	Runs the Crossword start screen (scene "Crossword - Start"): the language chosen on the hub, a "Random puzzle" button and one card per category (Animals, Food and...
3 Crossword/Editor/CrosswordCategoryBuilder.cs	583	Editor tool that sets up the Crossword categories as real saved scenes, in the same look as the menus and the redesigned Crossword: - it archives the 30 current level...
3 Crossword/Editor/CrosswordDesignBuilder.cs	867	Editor tool that rebuilds a Crossword puzzle scene in the redesigned look, as real saved scene objects (so the design is visible in the editor before pressing Play)....
3 Crossword/Gameplay/CrosswordAnswerKey.cs	62	Debug helper: lists which data script the current puzzle came from (e.g. "Cebuano_Level5") plus every Across and Down answer, right in the Inspector, so a tester can...
3 Crossword/Gameplay/CrosswordBlackCell.cs	21	Marker added (at runtime, by CrosswordGridGenerator) to every black/blocked cell it instantiates, recording its grid position. Black cells otherwise have no script of...
3 Crossword/Gameplay/CrosswordCell.cs	190	One letter cell in the crossword grid, instantiated by CrosswordGridGenerator - one per grid position that a word's letters actually cover (black/blocked cells use a...

Script	Lines	Purpose
3 Crossword/Gameplay/CrosswordCluePanel.cs	316	Builds and maintains the ACROSS/DOWN clue lists (one CrosswordClueRow per word, see CrosswordClueRow.cs) plus the active-clue banner shown above the grid. Owned and...
3 Crossword/Gameplay/CrosswordClueRow.cs	152	One clickable row in the Across/Down clue lists, instantiated by CrosswordCluePanel - one per CrosswordWordEntry. Shows the clue number and text, highlights itself...
3 Crossword/Gameplay/CrosswordGridGenerator.cs	240	Builds the visible crossword grid for one CrosswordPuzzle (see CrosswordPuzzleData.cs): instantiates a CrosswordCell for every grid position a word's letters actually...
3 Crossword/Gameplay/CrosswordInputController.cs	613	Orchestrates one crossword puzzle: tells CrosswordGridGenerator to build the grid, tells CrosswordCluePanel to build the clue lists from the result, then owns the...
3 Crossword/Gameplay/CrosswordPuzzleData.cs	52	Shared data types for the hardcoded per-level crossword puzzles in Assets/Main/Data (e.g. "cebuano - level 1.cs" through "tagalog - level 10.cs"). Each puzzle file...
3 Crossword/Gameplay/CrosswordPuzzleLibrary.cs	83	Looks up the hardcoded CrosswordPuzzle data (Assets/Main/Data/Crossword/Legacy levels/<Language>/ "<language> - level N.cs") for a given PuzzleLanguage + level...
3 Crossword/Gameplay/CrosswordTextStyle.cs	54	The ONE place that controls every text size in the crossword - grid letters, the small clue-number in the corner of each grid cell, and the Across/Down clue-list text...
3 Crossword/Gameplay/PanelPadding.cs	41	Reusable inner-margin control for any stretch-anchored UI panel (e.g. "crossword panel", "clue panel"). Attach it to a RectTransform whose anchors already stretch to...
3 Crossword/Level Loader/CrosswordLevelLoader.cs	130	Loads the 10 pre-made, pre-validated puzzles per language from puzzles_tagalog.json / puzzles_cebuano.json / puzzles_hiligaynon.json. Levels run 1 to 10: level 1 has...
3 Crossword/Level Loader/CrosswordPuzzleGenerator.cs	372	Builds crossword puzzles from the Tagalog / Cebuano / Hiligaynon word banks (fields: word, translation, definition, letters, language). IMPORTANT FIX vs. the previous...
4 Word Search/Editor/WordSearchDesignBuilder.cs	658	Editor tool that creates the two Word Search scenes as real saved scene objects (so the design is visible in the editor before pressing Play), in the same look as the...
4 Word Search/WordSearchBoardView.cs	346	The 10x10 letter grid of the Word Search play screen. It shows the letters (100 labels made in the scene by the design builder) and turns dragging into a selection:...
4 Word Search/WordSearchData.cs	128	The Word Search puzzle data model and its loader. The puzzles live in one JSON file per language, Assets/Main/Data/Word Search/Puzzles/...
4 Word Search/WordSearchGameController.cs	614	Runs the Word Search play screen (scene "WordSearch - Play"). It reads the chosen language, category, puzzle and mode from WordSearchSession, hides the puzzle's words...
4 Word Search/WordSearchGenerator.cs	160	Hides a puzzle's words in a 10x10 grid. Words run across, down or diagonally down-right (never backwards); words may cross where their letters match. The remaining...

Script	Lines	Purpose
4 Word Search/WordSearchHubTag.cs	68	The small tag on the Word Search card of the Language Selection hub (where Wordle and Crossword show "10 levels"). It shows how many Word Search puzzles the player...
4 Word Search/WordSearchSession.cs	146	Static hand-over between the Word Search scenes: which language, category and puzzle (A, B, C = 0, 1, 2) to play and in which mode. The language hub sets the...
4 Word Search/WordSearchStartController.cs	196	Runs the Word Search start screen (scene "WordSearch - Start"): the language chosen on the hub and one card per category (Animals, Food and drink, Body, Family and...
5 Library/LibraryController.cs	796	Runs the Library (dictionary) menu scene. At Start it builds the whole screen in code on top of the scene's existing background and back button, loads the three...
5 Library/LibraryData.cs	468	Pure-data side of the Library (dictionary) menu: parses the three dictionary JSON files (cebuano.json, hiligaynon.json, tagalog.json - native word, English...
5 Library/LibraryFavoritesStore.cs	68	Small static helper that reads and writes the Library's favorite words directly in PlayerPrefs, using the exact same key and format as LibraryData...
5 Library/LibraryLetterStrip.cs	32	Pointer handler on the slim A-Z strip at the right edge of the Library notebook. While the pointer is down or dragging over the strip it reports the vertical position...
5 Library/LibraryNotebook.cs	435	The "dictionary notebook": a scrollable A-Z word list on a ruled paper page with spiral rings, a red margin line and sticky-looking letter headers. It is virtualised...
5 Library/LibraryScrollTrack.cs	32	Pointer handler on the scroll bar track beside the Library notebook page. Clicking or dragging on the track reports the pointer's distance from the track's top (in...
5 Library/LibraryUiKit.cs	364	Helper set used by LibraryController / LibraryNotebook to build the Library screen from code: the shared color palette, sprites (rounded card, circle, star) and...
6 Learn/Editor/LearnSceneBuilder.cs	474	Editor tool that sets up the "learn the words" idea as real saved scenes. Menu: Tools > Learn > "Build Learn scene and buttons". - Creates "Learn - Cards"...
6 Learn/LearnController.cs	220	Runs the Learn screen (scene "Learn - Cards"): one card per word of a topic, so a new player meets the words before playing them. The card shows the word, its English...
6 Learn/LearnSession.cs	47	Static hand-over for the Learn screen (scene "Learn - Cards"): which game opened it, the language, the topic (category index) and the set (0, 1, 2 = A, B, C, the same...
6 Learn/LearnTopicButton.cs	33	Put on the "Learn" button of a topic card on a game's start screen (Wordle - Start, Crossword - Start, WordSearch - Start). Pressing it opens the Learn screen for...
7 Phrasebook/Editor/PhrasebookSceneBuilder.cs	565	Editor tool that sets up the Phrasebook and the Sentence builder as real saved scenes, in the look of the other menus. Menu: Tools > Phrasebook > "Build Phrasebook...

Script	Lines	Purpose
7 Phrasebook/PhraseBuilderController.cs	471	Runs the Sentence builder (scene "Phrasebook - Builder"): the player sees an English sentence and taps word tiles into the right order to say it in the language. The...
7 Phrasebook/PhraseCardsController.cs	183	Runs the phrase cards (scene "Phrasebook - Cards"): one card per phrase of a topic. A card shows the English line, the phrase in the language and, when the phrase has...
7 Phrasebook/PhraseData.cs	106	The Phrasebook's data model and loader. The phrases live in one JSON file per language (Assets/Main/Data/Phrasebook/phrases_<language>.json), found through...
7 Phrasebook/PhraseMenuButton.cs	25	Put on the Phrasebook card of the main menu (1 Main_menu), next to a Button on the same object. Pressing it opens the Phrasebook topics screen ("Phrasebook -...
7 Phrasebook/PhraseSession.cs	157	Static hand-over between the Phrasebook scenes ("Phrasebook - Topics", "Phrasebook - Cards", "Phrasebook - Builder"): which language, topic and phrase (all 0-based)...
7 Phrasebook/PhraseTopicsController.cs	159	Runs the Phrasebook topics screen (scene "Phrasebook - Topics"): a language selection at the top, like the tabs of the Library (Cebuano, Ilonggo, Tagalog; the pick is...
Audio/BackgroundMusic.cs	285	Plays the background music for every screen. When a scene with a different track loads, the old track stops right away (0.08 s anti-click fade) and only then does the...
Audio/Editor/TapToHearInstaller.cs	328	Editor tool that puts the "Tap to hear" buttons (WordAudio) into the scenes. Menu: Tools > Audio > "Add Tap to hear buttons". - draws the speaker icon sprite...
Audio/SoundEffects.cs	145	Plays a sound for every button in the game. It creates itself before the first scene loads (RuntimeInitializeOnLoadMethod), survives scene loads (DontDestroyOnLoad)...
Audio/WordAudio.cs	81	"Tap to hear": plays the recording of a word or phrase. A recording is an audio file in Assets/Main/Resources/Audio Words/<Language>/ (Cebuano, Ilonggo, Tagalog),...
Audio/WordAudioPlayer.cs	108	The object behind WordAudio: one AudioSource that survives scene changes plays the word recordings, and a small message that appears on top of any screen says "No...
Core/DictionaryEntry.cs	29	Data class for one dictionary entry (word, definition, part of speech).
Core/DictionaryManager.cs	104	Singleton (DontDestroyOnLoad) that loads and stores several dictionaries.
Core/Editor/ImageContainerDrawer.cs	82	Custom Inspector for ImageContainer, mirroring AlphabetDataDrawer (Assets/Sub/Word_Search/Scripts/Editor/AlphabetDataDrawer.cs): one ReorderableList per state...
Core/ImageContainer.cs	59	Reusable ScriptableObject data asset holding letter-to-sprite pairs (e.g. 'A' => "Alphabet A image.png") for the four common visual states an image/tile can be in:...
Core/LevelDataRegistry.cs	72	Points the games to their per-language level files, so those files can live next to each game's scripts instead of in a Resources folder. The one asset,...

Script	Lines	Purpose
Core/PlayerData.cs	39	Plain serializable data model for one player's save data: identity plus per-game level-completion progress. This is the schema persisted to disk by PlayerDatabase. To...
Core/PlayerDatabase.cs	239	Singleton persistence manager for the player's save data (PlayerData): loads it from disk on startup, exposes read/write access to identity and each game's progress,...
Core/SceneNavigator.cs	146	Centralized scene-navigation helper for the menu flow. Attach one instance to a "SceneManager" GameObject in each menu scene and wire UI Button OnClick events to its...
Core/ScreenFit.cs	207	Keeps the game the right size on any window or screen. It creates itself before the first scene loads (RuntimeInitializeOnLoadMethod) and survives scene loads, so...
Core/StreakTracker.cs	82	Static helper that keeps the player's day-streak in PlayerPrefs (no permissions needed). Call RecordPlay() whenever the player starts a minigame (SceneNavigator does...

Appendix B. Resources and data inventory

Location	Contents
Resources/LevelDataRegistry.asset	Registry: 4 lists of 3 TextAssets (Wordle, Crossword, Word Search, Phrasebook)
Resources/Audio Soundtrack	7 music tracks
Resources/Audio SFX	25 sound effects + README - Sound Effects.md
Resources/Audio Words	Empty (README only), one folder per language
Data/Language Dictionary	cebuano.json, hiligaynon.json, tagalog.json (63,540 entries)
Data/Wordle/Levels	wordle_levels_cebuano / ilonggo / tagalog .json (42 levels each)
Data/Wordle/Settings	WordleTextStyle, WordleTimerSettings, WordleSceneTimerDurations, WordleDialogPanelLayout (ScriptableObjects)
Data/Crossword/Puzzles	crossword_puzzles_<language>.json (21 puzzles each)
Data/Crossword/Settings	CrosswordTextStyle, CrosswordTextStyleV2 (ScriptableObjects: text sizes, cell border colour)
Data/Word Search/Puzzles	wordsearch_puzzles_<language>.json (21 puzzles each)
Data/Phrasebook	phrases_<language>.json (18, 17 and 18 phrases)
Data/*/Legacy*	Old per-level data (C# files and JSON) from the first designs, kept for reference
Fonts	Roboto 9 weights as SDF assets
Prefabs	11 prefabs (step 17)
Sprites	151 PNG files (step 7)
Scenes	15 playable scenes; Archive folders hold 128 older scenes

Appendix C. Editor tools (Tools menu)

Menu item	Creates or does
Tools > Wordle > Build category scenes and hub tag	Wordle - Start, Wordle - Play, hub tag
Tools > Wordle > Rebuild design in open scene / in all 30 level scenes	Re-applies the Wordle design (older per-level scenes)
Tools > Crossword > Archive current crossword and build category scenes	Crossword - Start, Crossword - Play, hub tag; archives the old level scenes
Tools > Crossword > Rebuild design in open scene / in all 30 level scenes	Re-applies the Crossword design
Tools > Word Search > Build Start and Play scenes	WordSearch - Start, WordSearch - Play
Tools > Learn > Build Learn scene and buttons	Learn - Cards and the Learn buttons
Tools > Phrasebook > Build Phrasebook scenes and menu card	The three Phrasebook scenes and the menu card
Tools > Settings > Build gear button and settings window	Gear chip and settings window on the main menu
Tools > Profile > Rebuild design in open scene / Archive current design and rebuild	The Profile screen
Tools > Audio > Add / Show / Hide Tap to hear buttons	Speaker buttons (WordAudio)

Recommended order of the builders on a fresh project: Wordle, Crossword, Word Search (so the hub tags and start screens exist), Learn (adds buttons to those start screens), Phrasebook, Settings, Profile, then Audio only if recordings exist.